PAMANTASAN NG LUNGSOD NG MAYNILA
(University of the City of Manila)
College of Information System and Technology Management
Computer Science Department

Bachelor of Science in Computer Science (BSCS)

S.Y. 2024-2025

CSC 0322 - COMPILER DESIGN

[CODE SLAM]

Group Leader:
Liwag, Justin E.

Group Members:
Balaoro, Clarisse Anne D.
Buscas, Louweesiana M.
Carlos, Michael D.
Carlos, Vince Anthony S.
Tendilla, Shieryl E.
Verdillo, Korinne L.
Villanueva, Maisie Allena F.

Submitted by:
CodeSlam

Submitted to:
Prof. Leisyl M. Mahusay
Prof. Jonathan C. Morano

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
STRUCTURE OVERVIEW **4**
Code Structure 5
GENERAL RULES **8**
DETAILED RULES **10**
II. Data Type and Literals 10
A. Feint Data Type 11
A. Floater Data Type 11
B. Chant Data Type 13
C. Stat Data Type 14
III. Type Conversion 14
IV. Type Casting 15
A. Feint Type Casting 15
B. Floater Type Casting 16
C. Chant Type Casting 17
D. Stat Type Casting 17
V. Chant Concatenation 18
VI. Data Structures 18
Rules for Array Declaration 21
VII. Variables 24
VIII. Constants 25
IX. Operators 26
X. Operator Precedence 29
XI. Expressions 30
A. Arithmetic Expressions 30
B. Relational Expressions 32
C. Logical Expressions 33
XII. Functions 35
A. Dunk Function Calls 37
B. Non-Dunk Function Calls 38
XIII. Statements 38
A. Rebound Statements 38
B. Inbound and Announce Statements 39
C. Conditional Statements 41
Switch-Get Statements 43
Rules for Switch-Get Statements 43
D. Looping Statements 44

1. Forward Loop Statement..... 44
2. Play Loop Statement..... 46
3. Run-Play Statement.....47
4. Timeout Statement..... 48
5. Resume Statement.....49
D. Assignment Statement.....50
XIV. Comments..... 51
PRE-DEFINED WORDS (RESERVED WORDS)..... 52
Data Types..... 52
Input and Output Statements.....52
Conditional Statements..... 52
Looping Statements.....53
Others.....54
RESERVED SYMBOLS..... 54
Arithmetic Operators.....54
Assignment Operators.....55
Relational Operators.....56
Logical Operators.....56
REGULAR DEFINITION..... 57
REGULAR EXPRESSION..... 60
DATA TYPES..... 60
Input and Output Statements.....60
Conditional Statements..... 60
Looping Statements.....60
Others.....61
B. Reserved Symbols..... 61
Arithmetic Operators.....61
Assignment Operators..... 61
Logical Operators.....62
Relational Operators.....62
Others.....62
C. Literals..... 63
TRANSITION DIAGRAM..... 64
Reserved Words.....64
Reserved Symbols.....66
Identifiers..... 68
CONTEXT FREE GRAMMAR.....73

STRUCTURE OVERVIEW

```
# Global declarations (variables, constants, etc.)

<global_declarations>;
<data_type> <variables> = <initial_value>; / # Feint @intNum = 5;
<datatype> <identifier>[<array_size>] = <array_initialization>; /
<constants> <data_type> <variable> = <initial_value>; #Const Floater @pi= 3.14;

#Function Declaration
<data_type> <identifier>(<parameters>);

#Function definitions
<function return_type> <function_name>(<parameters>) {
   # Local declarations
   <local_declarations>;

   # Statements (logic, calculations, etc.)
   <statements>;

   # Return value (if not void/Feint)
   Rebound <return_value>;
}

# Team Function
Team() {
   # Local declarations - arrays, variables, and constants (Orders does not matter in declarations)
   <local_declarations>;
   <variables> = <initial_value>; #Optional initialization

   # Statements (Orders does not matter in statements)
```

```
  <Assignment statements>; / <Inbound statements>; / <Announce statements>; /
  <Conditional statements>; / <Loop statements>;

  # Function Call (to other functions)
   <function_name>(<arguments>);
}
```

Code Structure

The program begins in a set of global declarations such as variables, constants, arrays, and functions. Global declarations are followed by the function declaration and function definition. The program runs from top to bottom, starting from global declarations and ends at the Team() function. The function contains local declarations and assignments to demonstrate its logic. The global declaration can be accessed inside the function definition and team function. The system utilizes curly braces to define code blocks, and each statement must end with a semicolon (;).

A. Declarations

Declaration specifies the meaning of an identifier such as its type and properties. It is used for variables, constants, functions, and arrays.

1. **Global Declarations**

   Global declarations are always defined at the beginning of the program and outside the functions. It can be accessed and utilized anywhere within the program. Global declarations can be a:

   a) **Variable** stores a piece of information;
   b) **Constant** an immutable variable;
   c) **Array** stores multiple data**;**

2. **Local Declarations**

   Local Declarations are always defined inside and at the beginning of a function. These declarations can only be accessed in the specific function where it is defined. If a variable is declared both globally and locally, the local declaration takes precedence.

B. Functions

Functions are a set of instructions that performs a specific task. Functions are used to group a series of commands into one unit, making it easier to reuse code and organize programs

better. Functions can only be created at the global level, functions inside other functions (nested functions) are not allowed.

1. **Team Function**

   The Team function is a predefined function that serves as the point of the program where all functions called inside are executed. The reserved word Team followed by parenthesis () is used to define the Team function and it cannot be reused inside any other function. User-defined functions can be utilized inside the Team() function.

2. **User-defined Functions**

   User-defined functions are functions created by the programmer**.** These functions are defined using the syntax: <data_type> identifier(<parameters>) { <statements >}. User-defined functions can be accessed and used within the Team function or other functions.

C. Assignments

1. **Assignment statements** are statements that assign the data value of an identifier.
2. **Announce statements** (print) are statements that display any processed outputs of the program.
3. **Inbound statements** (input) are statements that read user inputs.
4. **Conditional statements** execute a block of code if the condition is met.
5. **Loop statements** allow repeated execution of a block of code.
6. **Timeout statements** (break) initiates the exit in the loop statement.
7. **Resume statements** (continue) initiate moving onto the next iteration of the program.
8. **Function calls** are statements used to call a specific function for it to be executed by the program.
9. **Rebound statements** (return) are statements that indicate what value the program must return.
10. **Variable declaration statements** are used to define the data type and name of a variable.
11. **Variable initialization statements** are used to initialize a variable and its value.

D. Comments

Comments are a set of characters used in the program to enhance its readability, however, it does not have any effect in the program and it is ignored by the system. Comments

can either be a single-line comment which occupies only one line or multi-line comment that occupies multiple lines.

E. Program Termination

The program runs from top to bottom, where each line is processed sequentially. The global declarations are the first one to be processed followed by the function declarations and then the function definitions up to the Team() function. The program is terminated after the last statements in the Team function.

GENERAL RULES

1. Identifiers begin with an @ symbol followed by a lowercase letter and can contain any combination of lowercase and uppercase letters, digits, and the special character underscore "_", which is used for spacing.
2. Identifiers must be between 1 to 20 characters long to ensure clarity and ease of understanding for both developers and the compiler.
3. Identifiers are **case-sensitive**, meaning any difference in capitalization creates distinct identifiers.
4. Global declarations for variables, constants, and arrays, regardless of order, must be made before the **Team** function and any subfunctions.
5. Local declarations are variables defined within a function. These can only be accessed inside the function where they are declared.
6. Always specify the data type in declarations (e.g., Feint @myNumber = 5;).
7. All variables must be declared before use. A declared variable that is not used will be ignored during execution.
8. Declare only one type of variable per line (e.g., Feint @a, @b = 10; is accepted, but not Feint @a; Chant @str = "hello"; on the same line).
9. Constants are immutable once defined. They are declared using the reserved word Const followed by a variable.
10. Each data type has specific formatting rules for values:
    a. **Floater (float)** values are floating-point numbers storing up to 13 digits and 6 decimal places, rounding any extra decimal places.
    b. **Feint** (int) values must be whole numbers. It can store up to 13 digits.
    c. **Chant** (string) values must be enclosed in double quotation marks (" "), representing sequences of characters. Single quotes (' ') are not accepted.
    d. **Stat** (bool) values must be either **Win** (true) or **Loss** (false), representing boolean data.
11. A **tilde ("~")** is used as a negative symbol and to negate Feint and Floater Literals.
12. In type casting, there should be no space between the data type and parenthesis.
13. All reserved words must start with an uppercase letter followed by lowercase letters. Ensure the correct capitalization for valid data types except for logical operators (**AND, OR, NOT**) and that must be in all uppercase letters.

14. Reserved word **Inbound** is used for input statements, while **Announce** is used for output/print statements.
15. Function prototype and function definition must be declared before the **Team.**
16. The array size is static, meaning their size is fixed and cannot be changed after declaration. For clarity, the size and dimensions must match its elements
17. Arrays follow zero-based indexing, which implies that the number 0 is used to access the array's first element, the number 1 to access its second member, and so on.
18. The type of elements in an array is determined by its declared data type, meaning all elements must be of the same type.
19. 2-dimensional arrays are supported in the programming language.
20. All statements must end in a semicolon (;) signaling termination and separation from other statements.
21. Operations in expressions are performed following the PEMDAS rule: Parentheses, Exponents, Multiplication/Division (left-to-right), and Addition/Subtraction (left-to-right).
22. Values with different data types can be directly concatenated with each other except for a value with a Stat (boolean) data type, unless it was type casted.
23. Single-line comments start with “#”, and multi-line comments are enclosed within matching “|**|”. Therefore, any code or program written following “#” or enclosed within matching “|**|” will be considered as a comment.
24. Escape sequences like (\n, \t, \", \&, and \\) are permitted within Chant literals to represent non-printable characters.
25. Reserved words cannot be used as identifiers. However, if a reserved word uses different letter casing and is prefixed with the “@” symbol, it can be used as an identifier.

DETAILED RULES

I. Identifiers

An **identifier** is a name used to identify a variable, function or other user-defined item. Identifiers are essential because they give meaning to different elements of code, allowing programmers to reference them throughout the program.

Rules for Creating Identifiers

1. Identifiers begin with a @ symbol followed by a lowercase letter and could include a combination of lowercase and uppercase letters, digits, and the underscore (_) as the only special character.
2. Identifiers must be at least 1 character long and no more than 20 characters long.
3. Identifiers are case-sensitive, meaning small differences in capitalization create unique identifiers with different meanings within the program.

Valid Identifier	**Invalid Identifier**	**Reason**
@player	_player, $name, #name	Must not start with a special character except "@".
@d8te	@8ate	Must not start with a number.
@sHoot	@Shoot	Must not start with a capital letter.
@buzzer_Beater	@Buzzer Beater	Must have no spaces between separate words.

II. Data Type and Literals

l. **Data Types** refer to the classification of data that tells the compiler or interpreter how the programmer intends to use the data. Each data type specifies the type of operations that can be performed on the data and the way the data is stored in memory

II. A **literal** is a fixed value that appears directly in the source code of a program. These values represent data that is interpreted as-is during execution. Literals are written directly as part of the code and have a specific data type.

A. Feint Data Type

- **Feint** is used for storing integer values.

Rules for Feint

1. **Feint** can store integer values from -9999999999999 up to 9999999999999.
2. When a **Feint** starts with zero (0) followed by another number other than zero, the zero will be ignored and the preceding number is read.
3. A tilde (~) in front of a **Feint** is considered as the negative symbol.
4. Negative zero is prohibited (e.g. -0 or -0000 is prohibited but -0001 is allowed).

Valid Feint Literal	**Invalid Feint Literal**	**Reason**
23	"23"	Quotation marks (" ") are not an accepted **Feint** character.
~30	30~	Misplaced tilde symbol (~).
40474	40,474	Comma (,) is not an accepted **Feint** character.
35	35.5	Period (.) is not an accepted **Feint** character.
0	LeBron	Letters are not an accepted **Feint** character.

A. Floater Data Type

- **Floater (float)** is used for storing real numbers with decimal points.

Rules for Floater

1. **Floater** can store values ranging from -9999999999999.999999 up to 9999999999999.999999.
2. **Floater** must include a decimal point (.) between the whole number from the fractional part (e.g., 2.3)
3. **Floater** must be represented with a fractional part, even if it corresponds to a whole number (e.g. 3 should be 3.0).
4. **Floater** must include a whole number before the decimal point because a number before a decimal point is expected for it to be considered as floater. (e.g., .3 is not allowed, but 0.3 is valid).
5. If a **Floater** value exceeds 6 decimal places, it is rounded off to the nearest value. (e.g.: 6.6666666 = 6.666667)
6. A tilde (~) in front of a **Floater** is considered as the negative symbol.

Valid Floater Literal	**Invalid Floater Literal**	**Reason**
23.00	"23.00"	Quotation Marks (" ") are not a valid symbol for **Floater** literals.
~0.30	30.0!, #30.0, ?30.0	Invalid use of symbols.
40.474	40,474	A comma is not an accepted symbol for **Floater** literals.
24.0	240	The Given invalid example is classified as a **Feint** literal.
2.3	jordan	Non-numeric characters excluding tilde (~) and

		period (.) are not accepted as **Floater** literals.

B. Chant Data Type

- **Chant** is used for storing a sequence of one or more characters that may consist of letters, numbers, or symbols.

Rules for Chant

1. **Chant** can contain any of the ASCII characters.
2. **Chant** accepts empty strings (for example, "").
3. Use the (&) operator to concatenate **Chants**.
4. Escape sequences are character sequences that represent non-printable or restricted characters. These sequences start with a backslash (\), followed by one or more characters. The following escape sequences can be used inside a **Chant** literal:
    - \n - new line
    - \t - tab
    - \\ - \ symbol
    - \" - “” symbol
    - \& - & symbol

Valid Chant Literal	**Invalid Chant Literal**	**Reason**
"JORDAN"	'JORDAN'	Single quotation marks (' ') are not accepted when defining **Chant** literals.
"Anthony Davis"	"Nice	Unterminated **Chant**.
"10.5"	10.5	Floater literal is invalid as a **Chant**.

"passed"	"passed'	Unterminated **Chant**. A single quotation mark was used to terminate the **Chant**.
"Michael\n Jordan"	"Michael\z Jordan"	Invalid escape sequence.
"Lebron" & "James!"	"Lebron", "James!" "Lebron"&"James!"	Invalid use of a comma (,) as **Chant** operator.
"Hi, you'll be safe"	"Hi "you'll be safe!"	This results in a syntax error because the codeslam compiler sees two separate **Chant** ("Hi " and "you'll be safe!") with an incomplete statement in between.

C. Stat Data Type

- **Stat (boolean)** is used for storing **Win (True)** and **Loss (False)** values.

Rules for Stat Data Type

1. **Stat** must only be **Win** or **Loss**, other word/s are prohibited.
2. When **Stat** is printed, its value will output as either "1" for Win or "0" for Loss.

Valid Stat Literal	**Invalid Stat Literal**	**Reason**
Win	win	Invalid reserved word.
Loss	loss	Invalid reserved word.

III. Type Conversion

Type conversion is the process of converting a value from one data type to another. Essential for performing operations between different data types and for ensuring data compatibility.

Rules for Type Conversion

1. When an arithmetic operation involves both a **Feint** and a **Floater** number, the compiler will automatically convert the **Feint** number to a **Floater**.
2. When **Feint** or **Floater** literals, variables, or array elements are concatenated with a **Chant**, the compiler will automatically convert them, resulting in a **Chant.**
3. **Chant** is not accepted as an arithmetic operator.

Valid Type Conversion	**Invalid Type Conversion**	**Reason**
5 * 10.9	5 * “Ten”	**Chant** is not accepted as an arithmetic operator
"My age is " & 21	"My age is " + Win	**Stat** literal cannot be converted to **Chant**
@my_array[2] & “hello”	@my_array[] & “hi”	Array doesn’t have an element

IV. Type Casting

Type casting is the process of converting one data type into another. It is commonly used in programming to ensure that operations involving different types of data are handled correctly or to avoid errors.

Syntax

<data_type>(<value>);

where:

<data_type> = **Feint** , **Floater**, **Chant**, **Stat**

<value> = identifier, Feint literal, Floater literal, Chant literal, Stat literal

A. Feint Type Casting

Rules for Feint Type Casting

1. **Feint** may only be used for type casting if the **value** has a valid literal conversion to **Feint.**
2. When a **Floater** is converted to a **Feint** using type-casting, the decimal portion of the **Floater** is discarded, and only the whole number part is retained. No rounding is performed during this conversion.
3. When casting **Stat** literals into **Feint**, **Win** will be converted as 1, while **Loss** will be converted as 0.

Valid Feint Type Casting	**Invalid FeintType Casting**	**Reason**
Feint(“23”)	**Feint**(“twentythree”)	Words have no valid literal conversion to **Feint**
Feint(2.4)	**Feint** 2.4	Missing parentheses
Feint(Win)	**Feint**(win)	Invalid **Stat** literal

B. Floater Type Casting

Rules for Floater Type Casting

1. **Chant** may only be used for type casting if the contained **Chant** has a valid literal conversion to **Floater**
2. When casting **Feint** to a **Floater**, a decimal point and zero will be appended to the **Feint**, ensuring a valid **Floater** representation.
3. When casting **Stat** literals into **Floater**, **Win** will be converted as 1.0, while **Loss** will be converted as 0.0.

Valid Floater Type Casting	**Invalid Floater Type Casting**	**Reason**

Floater(“2.3”)	Floater(“two.three”)	Words have no valid literal conversion to Floater.
Floater(24)	Floater 24	Missing parentheses.
Floater(Loss)	Floater(loss)	Invalid **Stat** literal.

C. Chant Type Casting

Rules for Chant Type Casting

1. Any value can be type cast to **Chant**.

Valid Chant Type Casting	**InvalidChant Type Casting**	**Reason**
Chant(2.3)	Chant (2.3	Unterminated parenthesis.
Chant(24)	Chant 2	Missing parentheses.
Chant(Win)	chant(Win)	Invalid reserved word for **Chant**.

D. Stat Type Casting

Rules for Stat Type Casting

1. Values containing **Feint** or **Floater** can be typecasted to **Stat.**
2. Zero, regardless if it is a **Feint** or **Floater**, is always cast to **Loss**.
3. Any non-zero value (whether it's a positive or negative number) when subjected to **Stat** typecasting, will be cast to **Win**.
4. When casting **Chant** literals to **Stat**, empty **Chant** will be cast as **Loss**, while non-empty **Chant** will be **Win** regardless of the content as there is no type conversion from **Stat** to **Chant.**

Valid Stat Type Casting	**Invalid Stat Type Casting**	**Reason**
Stat(23)	Stat(23	Unterminated parenthesis.

Stat(~23)	Stat 23	Missing parentheses.
Stat(“”)	Stat (Five)	Invalid value. The given value is not a valid **Feint**, **Floater**, or **Chant** literal.

V. Chant Concatenation

- Chant concatenation involves adding or merging two or more Chants into one.

  Rules for Chant Concatenation

  1. If at least one operand is of type Chant, the “&” symbol serves as a string concatenation operator.
  2. Only **Feint** and **Floater** data types can be concatenated to **Chant**.
  3. To concatenate a **Stat** data type, type casting it either into **Feint** or **Float** is necessary to convert its value into **Chant**.
  4. The following data types can be concatenated using “&”. Two or more **Chants** can be concatenated directly.

Syntax

<chant_operand> & <operand>

<operand> & <chant_operand>

where:

<chant_operand> = values that contain a Chant literal

<operand> = Chant literal, Feint literal, Floater literal, type casts

VI. Data Structures

Data structures are specialized formats used to organize, manage, and store data efficiently. They provide a way to process and manipulate data depending on the task or algorithm at hand.

A. **Array**

An **array** is a data structure that stores a fixed-size, ordered collection of elements, all of the same data type. Arrays allow you to store multiple values in a single variable and access each value using an index.

Rules for Array Declaration

1. An array can be declared with or without initialization.
2. Array declarations start with a data type followed by an identifier, and the array size enclosed in square brackets [].
3. The array size must always be provided, and it can only be a positive Feint literal.
4. The minimum array size is 2 elements, and the maximum size is 999,999,999 elements.
5. Array size is static, meaning their size is fixed and cannot be changed after declaration.
6. Arrays should be declared with a data type, and the data type of the array should match the data typeof its elements.
7. Stat data type is not allowed for declaring arrays.
8. The size and dimensions of an array must match its elements.

Syntax

<data_type> identifier[<array_size>];

where:

<array_size> = Feint literal, identifier

Valid Array Declaration	**Invalid Array Declaration**	**Reason**
Feint @my_array[2];	$my_array[2];	Array is initialized without a proper identifier.
Floater @my_list[3];	@my_list[];	Array is initialized without an array size.

Feint @size = 3; Feint @num[@size];	Feint @size =; Feint @num{@size};	Invalid symbol for array size

Rules for Array Initialization

1. Array initialization starts with declaring the array and its size, followed by an equal sign (=), and then the elements enclosed in curly brackets {}, separated by commas.
2. The minimum array size must be two.

Syntax

<data_type> identifier[<array_size>] = {<array_elem>, <array_elem>};

where:

<array_size> = Feint literal

<array_elem> = Feint literal, Floater literal, Chant literal

Valid Array Initialization	**Invalid Array Initialization**	**Reason**
Floater @my_array[3] = {2.3, 2.4, 3.0};	Floater @my_array[3] = {2.3, 2.4};	Mismatched dimensions: the array expects 3 elements but 2 are provided.
Chant @my_list[3] = {"Michael", "Jordan", "!"};	Floater @my_list[3] = ();	Invalid use of parentheses.

Rules for Array Manipulation

1. Indexing can be used to retrieve individual elements from an array by simply stating the array name and the element index contained in square brackets "[]" (e.g., <array name>[<element index>]).
2. Arrays employ zero-based indexing, which implies that the number 0 is used to access the array's first element, the number 1 to access its second member, and so on.

3. Accessing an index outside the array's bounds will result in an error.
4. To update a specific element, use its index, followed by = and the new value.

Syntax

identifier[<element_index>];

identifier[<element_index>] = <array_elem>;

where:

<element_index> = Feint literal

<array_elem> = Feint literal, Floater literal, Chant literal

Valid Array Manipulation	**Invalid Array Manipulation**	**Reason**
@my_array[2] = 0;	@my_array[-2];	Accessing array elements that do not exist. Index for array elements must only be positive numbers greater than or equal to 2.
@my_array[2] = 3.0;	my_array[2] = "Three";	Missing @ symbol: The array must start with @ to follow the naming convention.

B. Two-dimensional Array

A 2D array is structured as a matrix, represented by rows and columns.

Rules for Array Declaration

1. Two-dimensional arrays are declared using a data type followed by an identifier followed by two sets of square brackets [][], indicating the rows and columns.
2. The array size of a two-dimensional array must always be provided.
3. Both the number of rows and columns must be provided as positive Feint literals or identifiers that store Feint values.
4. The minimum size for a two-dimensional array is 2x2, while the maximum size can be up to 999,999,999 x 999,999,999.
5. Like one-dimensional arrays, two-dimensional arrays have a fixed size that cannot be changed after declaration.
6. Stat data type is not allowed for declaring arrays.
7. The size and dimensions of an array must match its elements.

Syntax

<data_type> identifier[<array_size>][<array_size>];

where:

<array_size> = Feint literal, identifier

Valid 2D Array Declaration	**Invalid 2D Array Declaration**	**Reason**
Feint @my_array[2][3];	Feint $my_array[2][3];	Array is declared without a proper identifier.
Feint @my_list[3][2];	Feint @my_list[3][];	Array is declared without an array size.

Rules for Two-Dimensional Array Initialization

1. Two-dimensional arrays are initialized by specifying their size, followed by an equals sign (=) and enclosing the entire array in a set of curly braces ({}). Each row within the array is then enclosed in its own set of curly braces.

2. The minimum array size for initialization is 1x1.
3. The number of elements in each row must match the declared column size.

Syntax

<data_type> identifier[<array_size>][<array_size>]= {<array_elem>};

where:

<array_size> = Feint literal

<array_elem> = Feint literal, Floater literal, Chant literal

Valid 2D Array Initialization	**Invalid 2D Array Initialization**	**Reason**
Feint @my_array[2][2] = {{1,2}, {3,4}};	Feint @my_array[2][2] = {{1,2}}	Mismatched in rows: Array expects 2 rows.
Floater @my_grid[3][2]= {{1.5, 2.3}, {3.0, 4.1}, {5.2, 6.3}};	Feint @my_grid[3][2] = ({1, 2}, {3, 4, 5});	Invalid use of parentheses.

Rules for Two-Dimensional Array Manipulation

1. Access elements using row and column indices in the format identifier[row][column]. Both indices must be valid positive Feint literal.
2. Arrays use zero-based indexing, meaning the first row and first column are accessed using [0][0].
3. Attempting to access indices outside the declared row or column size results in an error.
4. To update a specific element, specify its indices followed by = and the new value.

Syntax

identifier[<element_index>][<element_index>];

identifier[<element_index>][<element_index>]= <array_elem>;

where:

<element_index> = Feint literal

<array_elem> = Feint literal, Floater literal, Chant literal

Valid 2D Array Manipulation	**Invalid 2D Array Manipulation**	**Reason**
@my_array[1][1] = 5;	@my_array[1][-1] = 5;	Column index must be a positive integer.
@my_array[0][0] = 3.5;	my_array[0][0] = 3.5;	Missing @ symbol: The array must start with @ to follow the naming convention.

VII. Variables

A **variable** in programming is a container that holds a value or data. Variables allow you to store data in memory and manipulate it throughout the execution of a program.

Rules in Declaration and Initialization of Variables

1. Variables declared within a block or function have local scope and are only accessible within that block or function.
2. Global variables, declared outside of any block or function, are accessible throughout the entire program.
3. Variable names must follow the identifier naming rules.
4. The equal sign (=) is used to assign values to variables (e.g., <var_Name> = value).
5. Multiple variables can be declared on a single line, without assigned values. They must have the same data types and must be separated by commas.

Syntax

<data type> <identifier> = <value>;

Where:

<data type> = Feint, Floater, Chant, Stat

<value> = Feint literal, Floater literal, Chant literal, Stat literal

Valid Variable Declaration and Initialization	**Invalid Variable Declaration and Initialization**	**Reason**
Feint @my_number = 23;	$my_number = "Jordan";	Incompatible variable initialization. The variable starts with “$”.
Floater @decimal_num = 2.3;	Floater @Decimal_num;	Invalid identifier. Identifiers must start “@” and follow with a lowercase.
Floater @decimal_num = 3.0;	Floater @decimal num = 3.0;	Incompatible variable initialization. The variable has a whitespace which is not allowed

VIII. Constants

Constants are immutable once defined, their value remains constant throughout the program execution. They are declared using the reserved word Const followed by a data type and a variable.

Rules for Constants

1. Constants are declared using the reserve Const followed by a data type and variable. Its value must always be initialized.
2. Constants can only be declared at a global declaration.
3. Constants are immutable and cannot be overridden even by a local variable.
4. The value stored by constants be of any type of literal including Feint literal, Floater literal, Chant literal, or Stat literal.

Syntax

Const <data_type> identifier = <literal>;

Where:

<literal> = Feint literal, Floater literal, Chant literal, Stat literal

Valid Constant Declaration and Initialization	**Invalid Constant Declaration and Initialization**	**Reason**
Const Floater @pi = 3.14;	const Floater @pi = 3.14;	Invalid reserved word for constants.
Const Chant @my_name = “Juan”;	Const Chant @my_name;	Value is not initialized.
Const Feint @totalStudents =35;	Const Feint totalStudents =35;	Invalid identifier.

IX. Operators

Operators are symbols or keywords used to perform operations on operands (data or variables). They are used in expressions to manipulate values and variables to produce a result. Operators are categorized based on the types of operations they perform.

- Below are the following types of operators:
    - Arithmetic Operators
    - Assignment Operators
    - Relational Operators
    - Logical Operators
    - Other Operators

Symbol	Name	Description
Arithmetic Operators		
+	Addition	Adds the operands from left to right
-	Subtraction	Subtract the operands from left to right
*	Multiplication	Multiplies the operands on both sides of the operator
/	Division	Divides the left operand by the right operand
%	Modulus	Returns the remainder of the division of the left operand by the right operand
Assignment Operators		
=	Assignment Operator	Assigns the left operand with the value of the right operand
+=	Addition Assignment Operator	Adds the right operand to the left operand and assigns the result to the left operand

-=	Subtraction Assignment Operator	Subtracts the right operand from the left operand and assigns the result to the left operand
*=	Multiplication Assignment Operator	Multiplies the left operand by the right operand and assigns the result to the left operand
/=	Division Assignment Operator	Divides the left operand by the right operand and assigns the result to the left operand
%=	Modulus Assignment Operator	Calculates the remainder of the division of the left operand with the right operand and assigns the result to the left operand
Relational Operators		
==	Is equal to	Checks if two operands, the left and right operands, are equal
!=	Not equal to	Checks if the left and right operands are not equal
<	Less than	Checks if the value of the left operand is less than the value of the right operand
>	Greater than	Checks if the value of the left operand is greater than the value of the right operand
<=	Less than or equal to	Checks if the value of the left operand is less than or equal to the value of the right operand
>=	Greater than or equal to	Checks if the value of the left operand is greater than or equal to the value of the right operand
Logical Operators		
AND	Logical AND	Returns **Win** if the value of both operands is **TRUE**
OR	Logical OR	Returns **Win** if at least one operand is **TRUE**

NOT	Logical NOT	Returns the opposite Stat value of the operand
Other Operators		
&	Chant Concatenation Operator	Concatenates Chant together when working with Chant data types
~	Negative Operator	Negates the numerical values, making a positive number negative and negative number positive
()	Grouping Operator	Used to group values or expressions

X. Operator Precedence

Precedence	Symbol	Description	Associativity
1	()	Parentheses	Left to Right
2	*	Multiplication	Left to Right
	/	Division	
	%	Modulus	
3	+	Addition	Left to Right
	-	Subtraction	
	~	Negative symbol	Left to Right
4	<	Less than	Left to Right
	<=	Less than or equal	
	>	Greater than	
	>=	Greater than or equal	

	==	Is equal to	
	!=	Not equal	
5	NOT	Logical NOT	Left to Right
	AND	Logical AND	
	OR	Logical OR	
6	=	Assignment	Right to Left
	*=	Multiplication Assignment	
	/=	Division Assignment	
	%=	Modulus Assignment	
	+=	Addition Assignment	
	-=	Subtraction Assignment	

XI. Expressions

- Combinations of operands and operators that result in a value when evaluated are called expressions.
- Expressions are grouped together and the evaluation order is managed using parentheses ().
- Expressions cannot make a statement by itself.
- Variables, literals, non-**Dunk** function calls, and typecasts as well as arithmetic, logical, and relational operations that evaluate to a value can all be used as expression operands.

A. Arithmetic Expressions

An **arithmetic expression** is a combination of numbers, operators, and sometimes parentheses, which evaluates to a single numeric value. Arithmetic expressions are used to perform mathematical operations such as addition, subtraction, multiplication, and division.

Rules For Arithmetic Expression

1. It must always include operands and arithmetic operators.
2. Arithmetic operands must be numerical values.
3. The correct operator precedence must be followed in arithmetic expressions.
4. An arithmetic operand can be any literal, variable, function return value, or other arithmetic expression, with the explicit restriction that only the Feint and Floater data types are permitted.
5. An arithmetic operand must be present at both the start and the end of an arithmetic expression.
6. Type casted **Feint** and **Floater** are accepted as arithmetic operands.
7. Every arithmetic expression within a statement must terminate with a semicolon (;).
8. Using arrays as operands is not permitted.

Syntax

<arithmetic_operand> <arithmetic_operator> <arithmetic_operand>

where:

<arithmetic_operand> = variable; <num_value>;<arithmetic_expr>; <num_typecast>

<arithmetic_operator> = +; -; *; /; %

Valid Expression	**Invalid Expression**	**Reason**

2 + 3.0;	2 ++ 3;	Invalid operator
@x * 3;	@x */ 3;	Invalid operator
(3.0) + 4;	Chant(5) + 4;	Invalid operand
@w - @x;	@w @x;	No operator
(@w * @x) % @y;	((@w * @x) + @y;	Unterminated parenthesis
(~7 + 8) * 9;	(7 + 8 * 9;	Unterminated parenthesis

B. Relational Expressions

- A relational expression is made up of two or more operands compared using relational operators, that gives a **Stat** result.

Rules for Relational Expressions

1. The outcome of a relational expression must be of the Stat data type, either **Win** (*true*) or **Loss** (*false*). The result is Loss if the operands do not satisfy the relational operator's condition; otherwise, it is Win.
2. It must consist solely of relational operators and operands of the same data type.
3. A relational operand can be a numeric literal, variable, or arithmetic expression, but a **Stat** or **Chant** operand is only valid when using ***==*** or **!=** with another operand of the same type.
4. Type casts can be used as relational operands, provided they are cast to match the data type of the other operand.
5. Arrays cannot be used as operands.
6. A relational expression must start with a relational operand, followed by a relational operator, and end with another relational operand.
7. Every relational expression in a statement must terminate with a semicolon (;).

Syntax

<relational_operand> <rel_op> <relational_operand>

where:

<relational_operand> = variable; <num_val>; <arithmetic_expr>; (<relational_operand>); <type_casted>; Chant literal; Stat literal;

<rel_op> = ==; !=; >; <; >=; <=

Valid Expression	**Invalid Expression**	**Reason**
@w < @x;	@w <;	Not a valid relational expression. The given example ends with an operator.
@age1 < @age2;	"LBJ" < 23;	Incompatible data types
@num1 <= @num2;	list1 << list2;	Invalid relational operator (<<)
"hi" != "Hi";	**Win** != "True";	Incompatible data types
Feint(5.0) >= 3;	**Chant**(5) >= 3;	Incompatible data types
3 * @x + 2 >= @y;	3 * @x + 2 = @y;	Not an expression
(@y <@z) >= @result;	(@y <@z >= @result;	Unterminated parenthesis
Win == **Loss;**	**Win** == 0;	Incompatible data type
Win != **Win;**	**Win** < **Win;**	Invalid relational operator

C. Logical Expressions

- A logical expression comprises logical operators and logical, numerical, or relational operands. It is utilized to assess conditions and generate a Stat result.

Rules for Logical Expressions

1. The result of **Stat** data type, which can either be **Loss**(false) or **Win**(true). The expression says **Loss** if the operand's value does not meet the operator's criteria, and **Win** otherwise.
2. It must consist exclusively of logical operators and logical operands.
3. Logical expressions are evaluated based on operator precedence.
4. Parentheses ( ) must be used to group expressions and manage the evaluation order when needed.
5. A logical operand can be a **Stat** literal, a variable holding a **Stat** value, a relational expression, or another logical expression.

Syntax

<logic_operand> <logic_operator> <logic_operand>

where:

<logic_operand> = <Stat_lit>; variable; <relational_expr>; <logical_expr>

<logic_operator> = AND; OR; NOT

Valid Expression	**Invalid Expression**	**Reason**

Win AND (@y < 10)	@x > 5 AND AND (@y < 10)	Invalid logical operator (AND AND)
(@a == @b) OR **Loss**	not (@a == @b) OR (@c != @d)	Invalid logical operator (NOT)
NOT (@x < 0)	NOT OR(y == 0)	Invalid logical operator (NOT OR)
((@x > 0) AND (@y < 5)) OR (@z == 3)	NOT (@x > 0, @y < 5)	Invalid (,) symbol
NOT ((@a OR @b) AND (@c OR @d) OR **Win**);	(@a OR OR @b) AND (@b OR @d);	Invalid logical operator (OR OR)

XII. Functions

- A function is a reusable block of code that performs a specific task. It takes inputs (arguments), processes them, and often returns an output. Functions help organize code, reduce redundancy, and improve readability by encapsulating logic in a defined structure.

Rules for Team Functions

1. The reserved word Team followed by parenthesis () is used to declare the Team function.
2. User-defined functions can be utilized inside the Team function.
3. There can only be one **Team** function in a program. It is not permitted to define **Team** more than once in a program; doing so will cause an error.

Rules for Creating User-Defined Functions

1. Function prototypes must be initialized before the **Team** function; initializing them afterward will cause an error.

2. To create a function prototype, start by indicating the return value's data type, followed by the function name, a parameter list, and end with a semicolon. Always include parentheses after the function name.
3. Function Definitions can be defined before the **Team** function. But can only be defined once.
4. Functions that are expected to return a value must provide a return value that matches the specified data type.
5. Enclose executable statements in curly braces ({}) for the function definition
6. Function names must adhere to identifier naming conventions.
7. Define parameters within the parentheses to accept input data, separating multiple parameters with commas (,).
8. Use the **Rebound** keyword to indicate the value that the function should return.
9. For functions that do not return a value, use the **Dunk** (void) keyword as the return type.
10. Functions inside other functions (nested functions) are not allowed.

Syntax

Function Declaration:
<data_type> identifier(<parameters>);
Function Definition:

<data_type> identifier(<parameters>) {
<statements>;
Rebound <value>;
}

Valid Function	**Invalid Function**	**Reason**

Floater @addition(Feint @x, Floater @y) {
@result = @x + @y;
Rebound @result;
}
	Feint @addition(@x, @y) {
	@result = @x + @y;
	}
		There is no Rebound statement

Dunk @hello() {
Announce("Let\'s Play!");
}
	Dunk @hello() {
	Announce("Let\'s Play!");
	Rebound 1;
	}
		There is a Rebound statement.

Chant @hello(){
Rebound “And One!”;
}
	Chant @hello(){
	@message = 4;
	Rebound message;
	}
		Function is expected to return a value of data type Chant, but received a value of Feint.

Dunk @shout(Chant @taunt, Feint @number);
Dunk @shout(Chant @name, Feint @jersey) {
Inbound(“Today! Meet ” & name & “ with jersey number is " & @jersey);
}
Team() {
@shout(“King James!”, 23);
}
	Dunk shout(Chant taunt, Feint number) {
	Inbound(“Today! Meet ” + name + “ with jersey number is " + jersey;
	}
	Team() {
	@shout(“Nieut Nieut! ”, 30);
	}
		The function is used before it’s declared.

A. Dunk Function Calls

When invoking a **Dunk** (Void) function, the return value doesn't need to be assigned or used in any expression; you can simply call it by writing its name followed by parentheses.

```
Dunk @greet() {
  Announce("Hello, World!\n");
}

Team() {
   @greet(); # Void function call
}
```

B. Non-Dunk Function Calls

In contrast, when calling a **non-Dunk** function, the return value must be utilized in an assignment or expression; failing to do so will lead to an error.

```
Feint @add(Feint @a, Feint @b) {
   Rebound @a + @b;
}

Team() {
   Feint @sum = @add(5, 3); # Non-void function call, using return value
  Announce("The sum is: " + @sum);
}
```

XIII. Statements

A. Rebound Statements

- **Rebound** (Return) statements are used within a function to indicate the value that should be returned when the function is called. A Rebound statement enables data to be sent back to the caller, terminating the function's execution and returning control to the calling function. Additionally, a Rebound statement cannot be left empty.

Rules for Rebound Statements

1. Return statements have to begin with the reserved word **Rebound** and then an expression.

Syntax Rebound <expression>; / Rebound <local_value>; Where: <expression> is of the same data type as the declared function's data type <local_value> is a locally declared variable.

Valid Rebound Statement	**Invalid Rebound Statement**	**Reason**
Rebound @var1;	Rebound @a+@b	Unterminated statement
Rebound @x + @y;	Rebound Announce@x;	Invalid expression

B. Inbound and Announce Statements

- A portion of the program that reads data from external sources, and displays information. It instructs the computer how to read and gather the information from an input device, and process and send the information to an output device.

- **Inbound** statements take input (e.i digits, characters, special characters, etc.) from the user.

- **Announce** statements display a specific message or any outputs processed by the program into the console or any output device.

Rules for Inbound Statements

1. The reserved term **"Inbound"** (input) must come first in input statements. This is followed by an open parenthesis, which contains a **Chant** where the user will type an input. The input prompt cannot be closed without the user pressing enter.

2. If a user provides an inbound with a wrong data type that does not match the expected type of the declared variable, the system will trigger an error, displaying a descriptive message to guide the user and, optionally, allow re-entry of the correct value to prevent runtime inconsistencies.

Syntax	**Example**
Inbound(<input_value>, <identifier>);	Inbound(”Enter ID: ”, @studentID);

Rules for Announce Statements

1. The reserved word **Announce** (print) must come first in output statements. This is followed by an open parenthesis. Next, **Chant** parameter, variable, array element, literal, function return value, or arithmetic, relational, or logical expressions, and finally, a closing parenthesis must come last.
2. Double quote marks ("") must separate **Announce** with Chant parameters. The **Announce** function has the capability to perform chant concatenation using the symbol “**&”**.
3. **Announce** Chant formatting adheres to compilers own escape sequences as specified in the Chant rules.

Syntax	Example
Announce(<value>);	Announce(@department);
Announce("<value>");	Announce("Hello, World!");
Announce(<expression>);	Announce(@x * 2);
Announce(“<value>” & variable);	Announce(“My name is “ & @name);
Syntax Announce(<value>);	
where: <value> = identifier, Feint literal, Floater literal, Chant literal, Stat literal, Array element, <expression> <expression> = <arithmetic_expr>; <relational_expr>; <logical_expr>	

C. Conditional Statements

Conditional statements (or control flow statements) are used to make decisions in programming. They allow the program to execute different code blocks based on whether a condition is true or false. Conditional statements help in controlling the flow of execution based on certain conditions.

Rules for Dribble-Pass-Shoot Statements

1. **Dribble (if)** statements must start with the reserved word **Dribble**, followed by a condition in parentheses and curly braces ({}) for the executable statements.
2. The **Pass (else if)** must start with the reserved word **Pass**, followed by a condition in parentheses and curly braces ({}) for the executable statements. However, **Pass (else if)**

can only be used in between **Dribble (if)** and **Shoot (else)** and can never be used as a standalone conditional statement.

3. **Dribble (if)** statements can either be a standalone statement or be paired with an **Shoot (else)** statement. If the condition in the **Dribble (if)** statement is not met, the **Shoot (else)** statement will be executed.
4. **Shoot (else)** statements must start with the reserved word **Shoot**, followed by a condition and curly braces ({}) for the executable statements.
5. Both **Dribble** and **Pass** statements must include a condition.

Syntax	**Example**

Dribble (<condition>) {
  <statements>
}
	Dribble (@score == 100) {
	  Announce(“Perfect Score!”);
	}

Dribble (<condition>) {
  <statements>
} Shoot (<condition>) {
  <statements>
}
	Dribble (@score >= 70) {
	  Announce("Passed!");
	} Shoot {
	  Announce("Failed!");
	}
	Dribble (@score >= 90) {
	  Announce("A");
	} Pass (@score >= 80) {
	  Announce("B");
	} Pass (@score >= 70) {
	  Announce("C");
	} Shoot {
	  Announce("F");
	}

Dribble (<condition>) { <statements>	Dribble (@score >= 70) { Announce("Passed!");

Syntax	**Example**

Dribble (<condition>) {
  <statements>
}
	Dribble (@score == 100) {
	  Announce(“Perfect Score!”);
	}

} Pass (<condition>) {
  <statements>
} Shoot {
  <statements>
}
	} Shoot {
	  Announce("Failed!");
	}
	Dribble (@score >= 90) {
	  Announce("A");
	} Pass (@score >= 80) {
	  Announce("B");
	} Pass (@score >= 70) {
	  Announce("C");
	} Shoot {
	  Announce("F");
	}

where: <condition> = <relational_expr>; <logical_expr>; Result_literal <statements> = <expr_stmt>; <func_call>, <Announce_stmt>; <Inbound_stmt>; <condi_stmt>	

Switch-Get Statements

- Switch-Get statements provide an alternative way to handle multiple conditional checks.

  Rules for Switch-Get Statements

  1. The **Switch()** statement evaluates an expression.
  2. The Get keyword checks if the expression matches a specific value.
  3. The default keyword provides a code block to execute if no Get matches the expression.

Syntax	**Example**

Switch (<identifier>, <feint>, <chant>) {
Get <value>:
<statements>
Timeout;
Get <value>:
<statements>
Timeout;
Get <value>:
<statements>
Timeout;
Default:
<statements>
}
	Chant @grade;
	Inbound("Enter your grade", @grade);
	Announce("Your grade is" & @grade);
	
	Switch(@grade){
	Get 'A':
	Announce("Excellent");
	Timeout;
	Get 'B':
	Announce("Very Good");
	Timeout;
	Get 'C':
	Announce("Good");
	Timeout;
	Get 'D':
	Announce("Fair");
	Timeout;
	Get 'F’:
	Announce("failed");
	Timeout;
	Default:
	Announce("Invalid Grade");
	}

Where: <value> = <feint, chant, const> <statements> = <expr_stmt>; <func_call>, <Announce_stmt>; <Inbound_stmt>; <condi_stmt>	

D. Looping Statements

- Looping statements allows repetitions of blocks of statements multiple times.

1. **Forward Loop Statement**

    - Statements in a **Forward (for)** loop iterate over a list, executing a block of statements for every item in the list.

    Rules for Forward Loop Statements

    a. The reserved word **"Forward"** must always start with a loop statement. Followed by a whitespace. Then, an opening parenthesis, counter initialization, loop condition, (+=) symbol or (-=) symbol, and a closing parenthesis must come next.
    b. Counters in **Forward** loops must always be of type Feint.
    c. Two curly braces need to encapsulate loop statements.

Syntax	**Examples**

Forward (counter initialization; condition; +=/-=) {
<stmts>;
}
	Forward (@i = 0; @i <=3; @i+=1) {
	Announce(“*”);
	}

	Forward (@counter = 3; @counter < @num3 + @num4; @counter += 1) {
	Announce((@num+3 < @num+4) > (@num+3 <= @num+3));
	Dribble (@num+3 < Chant(3.1)) {
	Announce(“Hello”);
	Dribble (@num1 < 5) {
	Announce(“Hello world ulit”);
	Timeout;
	} Pass (@num1 + @num2 AND @number)

	} }

	Feint @sum = 0;
	Forward (@num = 0; @num < 5; @num += 1){
	@sum += @num;
	}
	Announce("The sum of numbers 1 to 5 is " & @sum);

Where: <condition> = <relational_expr>; <logical_expr>, Stat lit <stmts> = <expre_stmt>; Timeout; Resume; <func_call>; <Inbound_stmt>; <Announce_stmt>; <condi_stmt>	

2. **Play Loop Statement**

- **Play (while)** loop statements execute a block of code repeatedly as long as the specified condition is true, making them especially useful when the number of iterations is not predetermined.

Rules for Play Loop Statements

a. While loop statements must start with the reserved word **Play**, followed by a whitespace and an open parenthesis. A condition inside the parentheses and the loop statements enclosed in curly braces.
b. To create a while loop, the following syntax must be followed:

```
Play (<condition>){
    <stmts>;
```

}

i. **Play** : Introduces the loop

ii. **<condition>** : The condition may be a **Stat** literal, a relational expression, a logical expression, or a combination of both relational and logical expressions.

iii. **"{"** Marks as the start of the loop body,

iv. **<stmts>** are the content of the code block executed. Valid content of <stmts> in Play loop are definitions and statements (output, conditional, iterative, insert, update, delete). Inbound (input) statement is not allowed inside the Play loop. However, the <stmts> inside the Play loop can be null.

v. **"}"** Marks as the end of the loop body.

Syntax	**Example**

Play (<condition>) {
  <stmts>;
}
	Feint @count = 1;
	Play (@count < 5) {
	  Announce(@count);
	  @count += 1;
	}

	Feint @x = 0;
	Play (@x < 5) {
	  Announce("Iteration " & @x);
	  @x += 1; # Increment @x to eventually end the loop
	}

	Feint @num = 0;
	Play (@num != 10){
	  Dribble(@num % 2 == 0){

	Announce(@num & “is even.”); } @num +=1; }
where: <condition> = <relational_expr>; <logical_expr>, Stat lit <stmts> = <expr_stmt>; Timeout; Resume; <Inbound_stmt>; <Announce_stmt>; <condi_stmt>	

3. **Run-Play Statement**

- The **Run-Play** is a loop statement used to repeat some part of the code till the given condition is fulfilled.

Rules for Run-Play Statements

a. A **Run-Play** loop starts with the keyword **Run** followed by a block of code enclosed in {}.
b. The block of code must be followed by the keyword **Play** and a condition enclosed in parentheses ().
c. The statement ends with a semicolon ;

Syntax	**Example**
Run { <stmts>; } Play (<condition>);	Run { Announce (@count); @count += 1; } Play (@count < 5);

4. **Timeout Statement**

- The **Timeout (break)** statement is used to exit a loop when a certain condition is satisfied.

Rules for Timeout Statements

a. The **Timeout** statement must not be used outside of loop constructs.
b. The **Timeout** statement immediately exits the loop, regardless of the loop's iteration conditions.

Syntax	**Example**

<Forward statement> {
<Dribble statements> {
Timeout;
}
<Announce Statement>;
}
	Forward (@i = 0; @i <10; @i+=1){
	Dribble (@i == 5) {
	Timeout; #exit the loop when i equals 5
	}
	Announce(@i);
	}

5. **Resume Statement**

- When a condition is met, the **Resume (continue)** statement is used to move on to the next iteration and skip the remaining code inside the loop for the current iteration.

Rules for Resume Statements

a. **Resume** statements must only be utilized inside of loops. It is not allowed to utilize the **Resume** statement outside of loop constructs.
b. The **Resume** statement skips the current iteration of the loop and proceeds to the next iteration.

c. There should be no whitespace between the reserved word **Resume** and the semicolon.

Syntax	**Example**

<Forward statement>{
<Dribble statements> {
Resume;
}
<Announce Statement>;
}
	Forward (@i = 0; @i <=3; @i+=1) {
	Dribble (@i == 3) {
	Resume;
	}
	Announce(“*”); #skip the following code (Announce) when i equals 3 and moves to the next iteration of the loop
	}

	Forward (@num = 1; @num < 10; @num += 1){
	Dribble(@num % 2 != 0){
	Announce(@num & “is an odd number”);
	} Shoot {
	Resume; #skips the even numbers.
	}
	}

D. Assignment Statement

- The **Assignment** statements are used to assign new values to existing variables and perform operations simultaneously.

Rules for Assignment Statement

a. Identifier: The name of the variable to which the value is assigned.
b. Assignment Operators: =, +=, -=, *=, /=, %=
c. The assignment statement applies the value on the right side of the expression to the variable on the left side.
d. Value/Expression: The value or result of an operation on the right-hand side of the assignment.

Syntax <data_type> <identifier> <assign_op> <value>;
where: <data_type> = Feint, Floater, Chant, Stat <assign_op> = =, +=, -=, *=, /=, %= <value> = Variable, Feint literal, Floater literal, Chant literal, Stat literal, <expression>

Valid Assignment Statement	**Invalid Assignment Statement**	**Reason**
Feint @num = 23;	Feint @Num = 23;	The variable starts with "@" and is followed by a capital letter.
Floater @average = (@grades * @units) /@ total_units;	Floater @average = (@grades * @units) /@ total_units	Unterminated statement
Feint @num += 23;	Feint @num =+ 23;	Invalid assignment operator
Chant @myPlayer = "Jordan";	Floater @total ==- 27.5;	Invalid assignment operator
Chant @letter = "M";	Floater @length = 6.5s;	Floating-point value cannot be assigned to a variable

		with a “s” in the end.

XIV. Comments

- Comments provide explanations, descriptions, or notes within the source code. They are ignored by the compiler or interpreter and are not executed, serving as documentation to enhance code readability.

Rules in Writing Single-line Comments

1. Single-line comments, created using the Pound Symbol (#) followed by the comment text.
2. Once the # symbol is encountered on a line, the subsequent characters will be ignored by the program. Single-line comment will end after a newline.
3. Single-line comments can be positioned either at the end of a line of code or on a dedicated line by themselves.
4. Comments should not be placed inside function parameters or within any uncompleted statements.

Rules in Writing Multi-line Comments

1. Multi-line comments in CodeSlam are created using a vertical bar |* at the start, followed by the comment, and another vertical bar to close it *|
2. If the |**| appears within the multi-line comment, the program will treat it as the conclusion of multi-line comments.

Syntax	**Example**
|* comment *|	|* This is a multi -line comment. *|
# comment	# This is a single-line comment.

PRE-DEFINED WORDS (RESERVED WORDS)

RESERVED WORDS	PYTHON OR C COUNTERPART	DESCRIPTION
Data Types		
Feint	int	Data type for whole numbers
Chant	str	Data type for a sequence of one or more characters that may consist of letters, numbers, or symbols.
Floater	float	Data type representing whole numbers and a decimal.
Stat	bool	Data type that holds the values for either Win or Loss.
Input and Output Statements		
Inbound	input	Reads and stores data.
Announce	print	Displays data as formatted.
Conditional Statements		
Dribble	if	Discerns whether a condition is True or False. If the evaluation holds true, it executes the statements within its body. Otherwise, it does not execute the statements within its body.
Shoot	else	Extension of an “if” statement. Provides the default statements to execute if all conditions are evaluated as False.
Pass	elif	Extension of an “if” statement. Offers an alternate condition, which can be determined to be True or False. If the evaluation holds true, it executes the statements within its body. Otherwise, it does not execute the statements within its body.

Switch	switch	Instead of writing many Dribble..Shoot statements, you can use the Switch statement.
Get	case	Each value is called a case, and the variable being switched on is checked for each switch case.
Default	default	The default reserved word specifies the default block of code in a switch statement, which is the code to run if there is no case match in the switch.
Looping Statements		
Forward	for	Iterates the statements within its body for a specified amount of times.
Play	while	Executes the statements within its body repeatedly while the condition is True.
Run	do	Indicates the start of a loop body that will execute at least once, regardless of the condition.
Run-Play	do/while	This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.
Timeout	break	Terminates the loop once a specified condition is met.
Resume	continue	A loop control statement that triggers the program to skip to the next iteration of a loop when executed.
Others		
Rebound	return	Used for exiting a function and returns a value to the caller if prompted.

Win	true	Stat value returned when an expression is evaluated as correct.
Loss	false	Stat value returned when an expression is evaluated as incorrect.
Team	Main	Refers to the function that serves as the entry point for program execution.
Dunk	void	Refers to functions that don't return any value.

RESERVED SYMBOLS

RESERVED SYMBOLS	**DESCRIPTION**
Arithmetic Operators	
+	Addition: Adds values together.
-	Subtraction: Finds the difference between values.
*****	Multiplication: Multiplies values by another.
/	Division: Divides values by another.

%	Modulus: Finds the remainder after division.
Assignment Operators	
=	Assignment: Assigns a value to a variable.
+=	Add and Assign: Adds and assigns the result to a variable.
-=	Subtract and Assign: Subtracts and assigns the result to a variable.
***=**	Multiply and Assign: Multiplies and assigns the result to a variable.
/=	Divide and Assign: Divides and assigns the result to a variable.
%=	Modulus and Assign: Finds modulus and assigns the result to a variable.
Relational Operators	
==	Equal to: Checks if two values are equal.
!=	Not equal to: Checks if two values are not equal.

<	Less than: Checks if one value is smaller than another.
>	Greater than: Checks if one value is larger than another.
<=	Less than or equal to: Checks if one value is smaller or equal to another.
>=	Greater than or equal to: Checks if one value is larger or equal to another.
Logical Operators	
AND	Logical AND: Returns True if the value of both operands are True.
OR	Logical OR: Returns true if at least one operand is True.
NOT	Logical NOT: Returns the opposite Boolean value of the operand.
Other Operators	
&	Chant Concatenation: Used for concatenation when working with Chant data types
[]	Array Size/Index: Used to specify the size or index of an array.

{}	Code Blocks: Used to define the start and end points of a programming structure (i.e. functions, loops, conditional statements or array initialization).
()	Grouping: Used to group values or expressions
~	Negative: Negates the numerical values, making a positive number negative and negative number positive
@	"@" sign is used to write or make a variable initialization. Every identifier must strictly start with an "@" sign.

REGULAR DEFINITION

NAME	**DEFINITION**
zero	0
digit	1, 2, 3, 4, 5, 6, 7, 8, 9
num	<zero>, <digit>
lowercase	a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z
uppercase	A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z
alpha	<lowercase>, <uppercase>
alphanum	<alpha>, <num>
Stat	Win, Loss

arithmetic_op	+, -, *, /, %
assign_op	+, +=, -=, *=, /=, %=
relational_op	==, !=, <, >, <=, >=
logical_op	AND, OR, NOT
other_op	&, [], {}, (), ~, @
ascii	space, !, ", #, $, %, &, ', (,), *, +, ,, -, ., /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, :, ;, <, =, >, ?, @, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, [, \,], ^, _, `, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z, {, |, }, ~
new_line	\n
tab	\t
space	␣
whitespace	<new_line>, <tab>, <space>
delim1	<whitespace>, (
delim2	<whitespace>, {
delim3	;
delim4	<whitespace>, <num>, (, @
delim5	<whitespace>, <alphanum>, “
delim6	<whitespace>
delim8	:
asval_delim	<whitespace>, (, @, <num>
chant_delim	<whitespace>, comma, ;,), }

equal_delim	<whitespace>, <num>, @, “,(
stat_delim	<whitespace>, ;,)
idt_delim	<white space>, comma, [, (,], ;,)
num_delim	<whitespace>, +, -, *, /, %, <, ;, space comma,),], =, !, (, }, :
other1_delim	<whitespace>, num, @
other2_delim	<whitespace>, =, ;
other3_delim	<whitespace>, ;, alphanum, }, comma
other4_delim	<whitespace>, @, ”, num
other5_delim	;,), {, }

Legend

* - null or many

λ - null/none

$^{+}$ - one or many

REGULAR EXPRESSION

A. Reserved Words

Reserved Words	**Regular Expression**	**Token**
DATA TYPES		
Feint	(F)(e)(i)(n)(t)	Feint
Chant	(C)(h)(a)(n)(t)	Chant
Floater	(F)(l)(o)(a)(t)(e)(r)	Floater
Stat	(S)(t)(a)(t)	Stat
Input and Output Statements		
Inbound	(I)(n)(b)(o)(u)(n)(d)	Inbound
Announce	(A)(n)(n)(o)(u)(n)(c)(e)	Announce
Conditional Statements		
Dribble	(D)(r)(i)(b)(b)(l)(e)	Dribble
Get	(G)(e)(t)	Get
Shoot	(S)(h)(o)(o)(t)	Shoot
Switch	(S)(w)(i)(t)(c)(h)	Switch
Pass	(P)(a)(s)(s)	Pass
Looping Statements		
Forward	(F)(o)(r)(w)(a)(r)(d)	Forward
Play	(P)(l)(a)(y)	Play

Timeout	(T)(i)(m)(e)(o)(u)(t)	Timeout
Resume	(R)(e)(s)(u)(m)(e)	Resume
Run	(R)(u)(n)	Run
Others		
Default	(D)(e)(f)(a)(u)(l)(t)	Default
Rebound	(R)(e)(b)(o)(u)(n)(d)	Rebound
Win	(W)(i)(n)	Win
Loss	(L)(o)(s)(s)	Loss
Team	(T)(e)(a)(m)	Team
Dunk	(D)(u)(n)(k)	Dunk

B. Reserved Symbols

Reserved Symbols	**Regular Expression**	**Token**
Arithmetic Operators		
+	(+)	+
-	(-)	-
*	(*)	*
/	(/)	/
%	(%)	%
Assignment Operators		

=	(=)	=
+=	(+)(=)	+=
-=	(-)(=)	-=
*=	(*)(=)	*=
/=	(/)(=)	/=
%=	(%)(=)	%=
Logical Operators		
AND	(A)(N)(D)	AND
OR	(O)(R)	OR
NOT	(N)(O)(T)	NOT
Relational Operators		
==	(=)(=)	==
!=	(!)(=)	!=
<	(<)	<
>	(>)	>
<=	(<)(=)	<=
>=	(>)(=)	>=
Others		
&	(&)	&
[]	([)(])	[]

{}	({)(})	{}
()	(()())	()
~	(~)	~
@	(@)	@

C. Literals

Literals	**Regular Expression**	**Token**
identifier	(@)(lowercaps)(alphanum/_/λ)^{18}	identifier
Feint Literal	(~/λ)(num)(num/λ)^{12}	FeintLiteral
Floater Literal	(~/λ)(num)(num/λ)^{12}(.)(num)(num/λ)^{5}	FloaterLiteral

D. Others

Literals	**Regular Expression**	**Token**
Single line comment	(#)(ascii)$^{+}$(new_line)	SingleCom
Multi-line comment	(|)(*)(ascii)$^{+}$(new_line)*(*)(|)	MultiCom

Legend

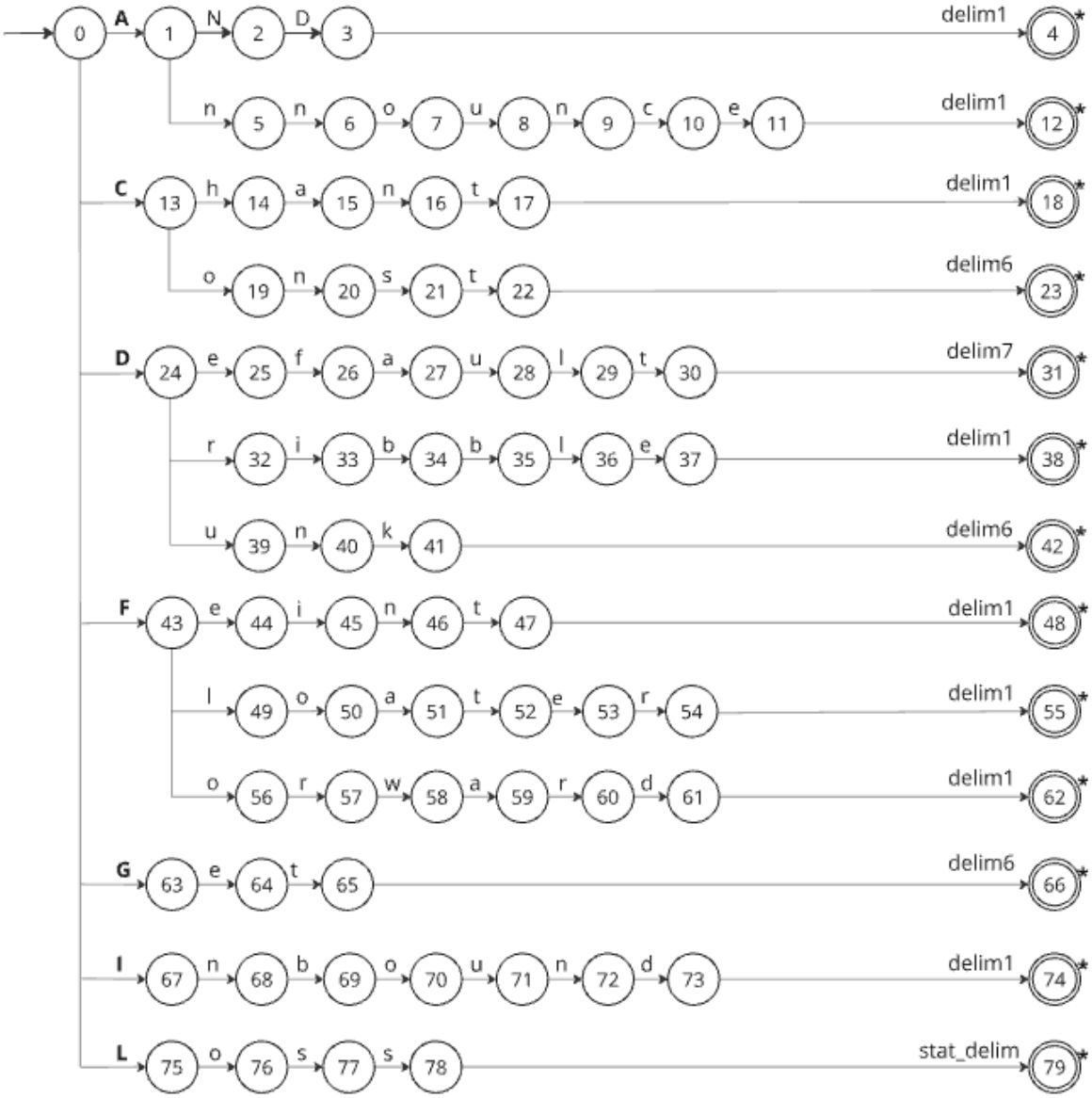
* - null or many

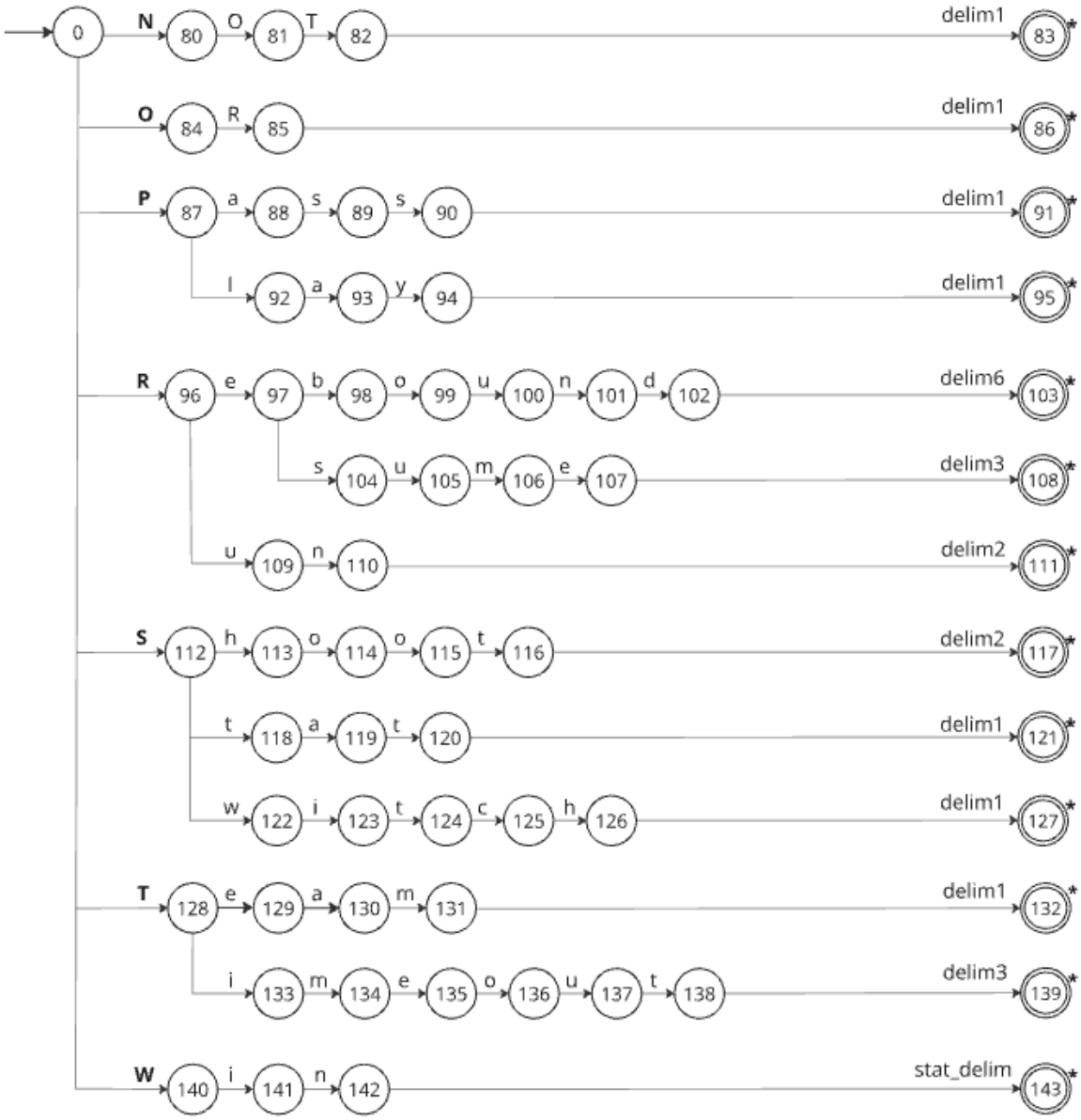
λ - null/none

$^{+}$ - one or many

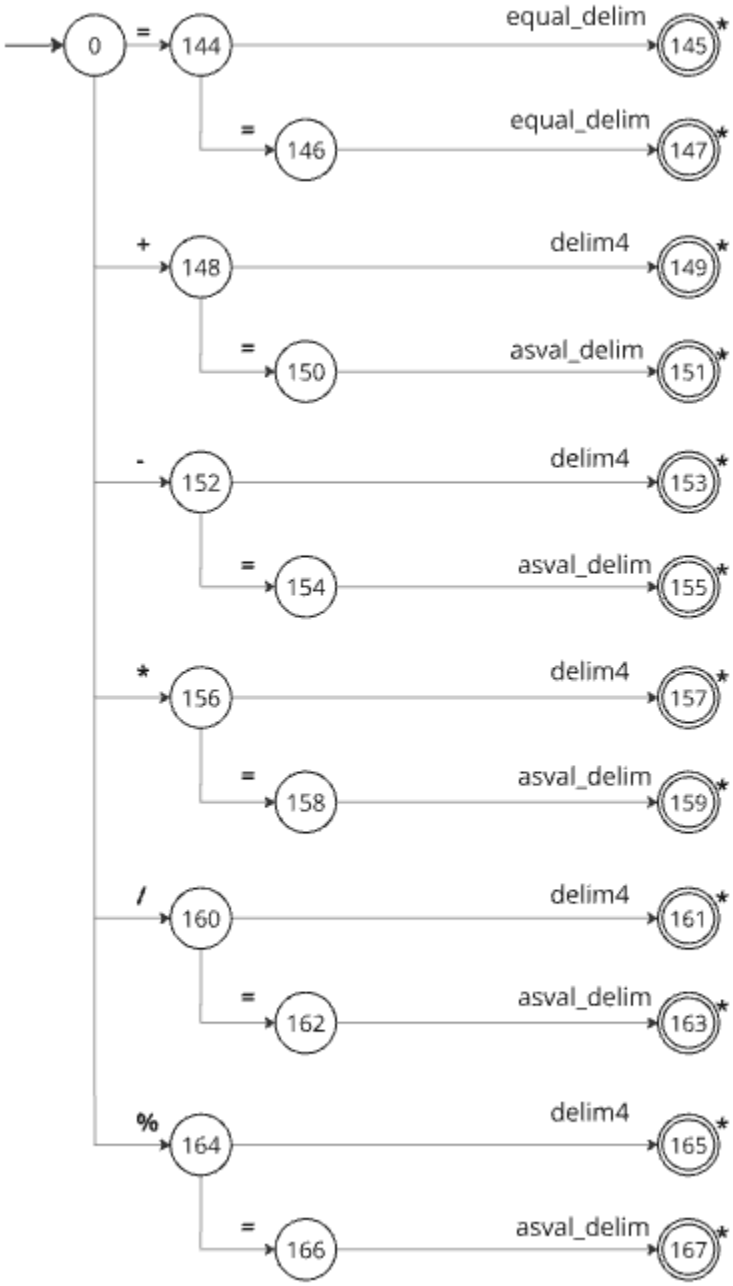
TRANSITION DIAGRAM

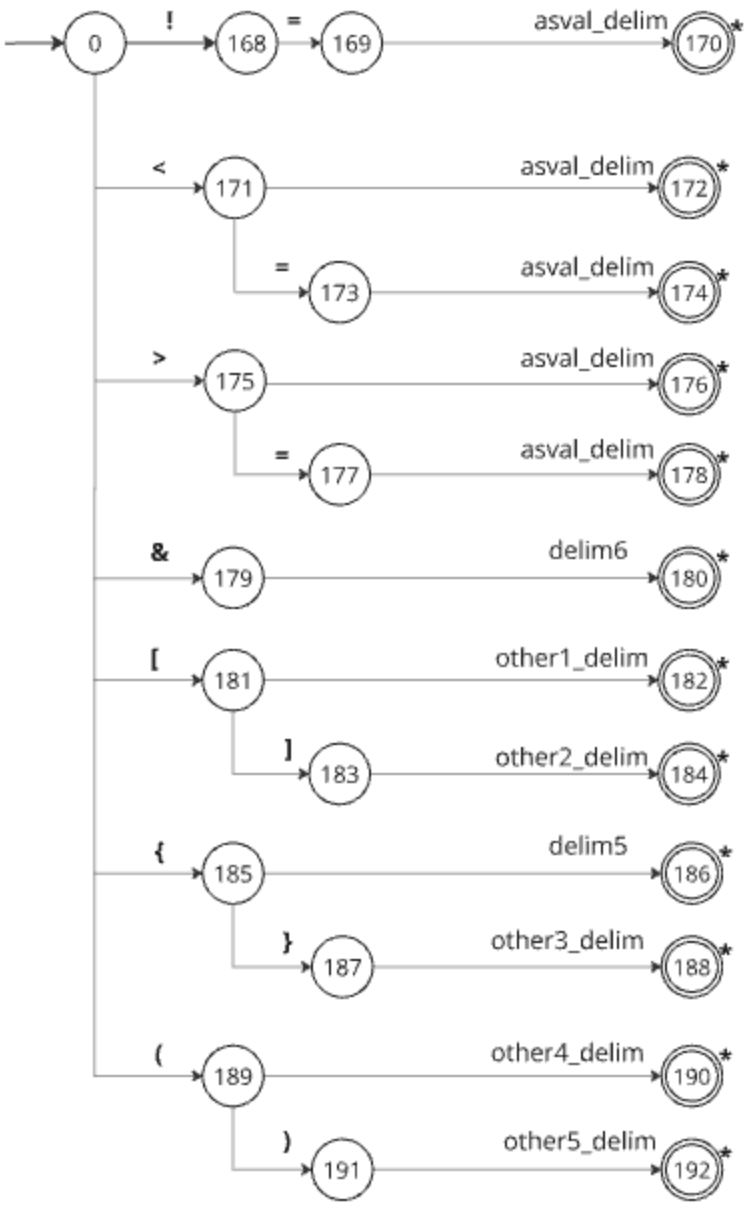
Reserved Words



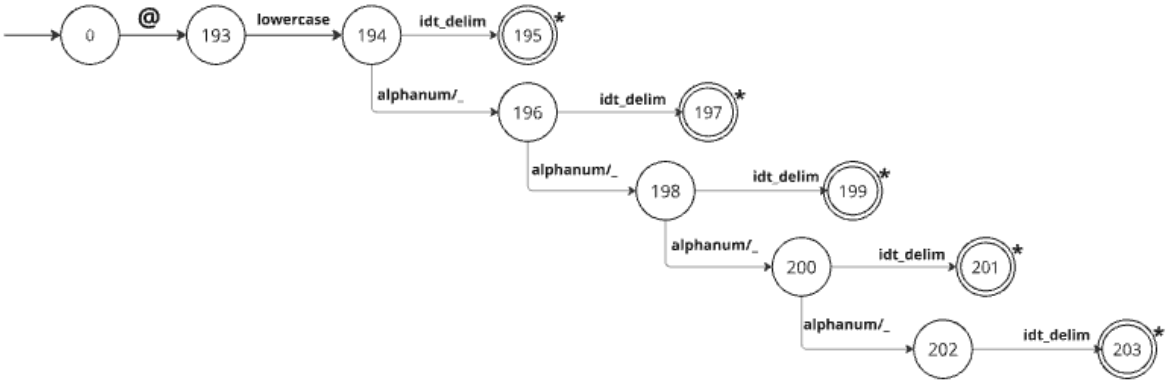

0
N
80
O
81
T
82
delim1
83
O
84
R
85
delim1
86
P
87
a
88
s
89
s
90
delim1
91
I
92
a
93
y
94
delim1
95
R
96
e
97
b
98
o
99
u
100
n
101
d
102
delim6
103
s
104
u
105
m
106
e
107
delim3
108
u
109
n
110
delim2
111
S
112
h
113
o
114
o
115
t
116
delim2
117
t
118
a
119
t
120
delim1
121
w
122
i
123
t
124
c
125
h
126
delim1
127
T
128
e
129
a
130
m
131
delim1
132
i
133
m
134
e
135
o
136
u
137
t
138
delim3
139
W
140
i
141
n
142
stat_delim
143

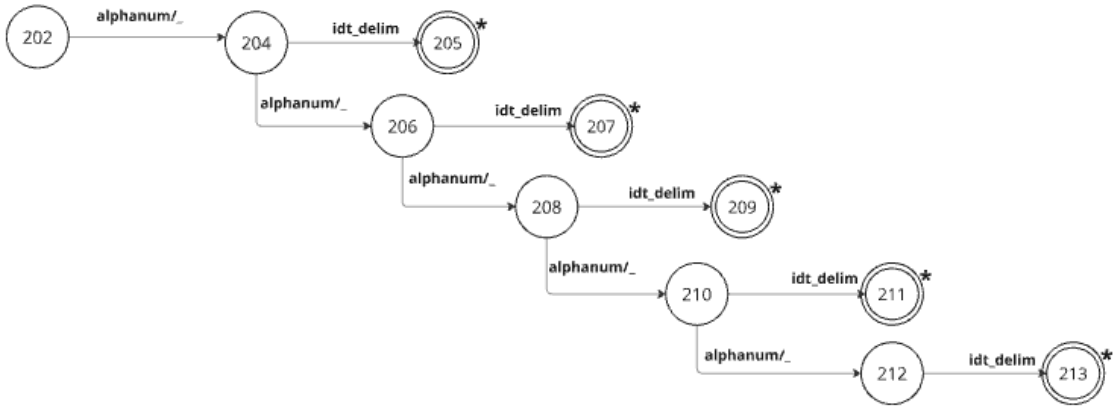
Reserved Symbols

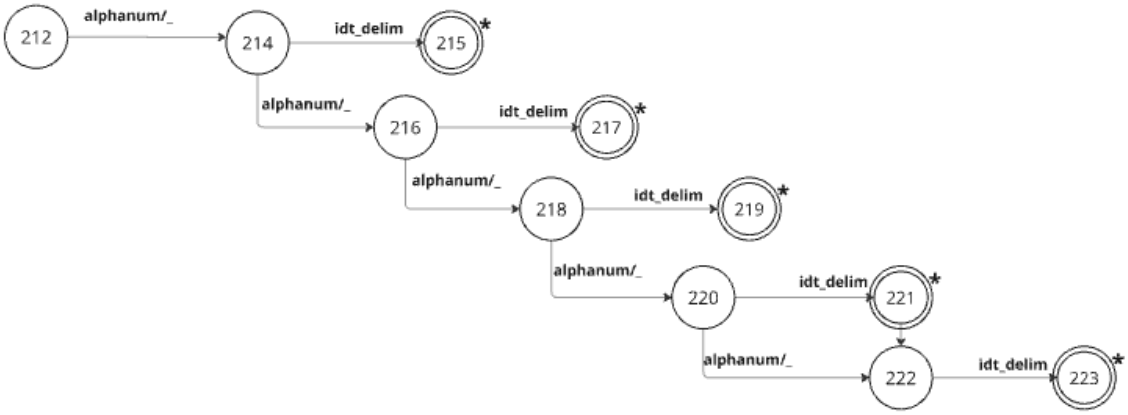


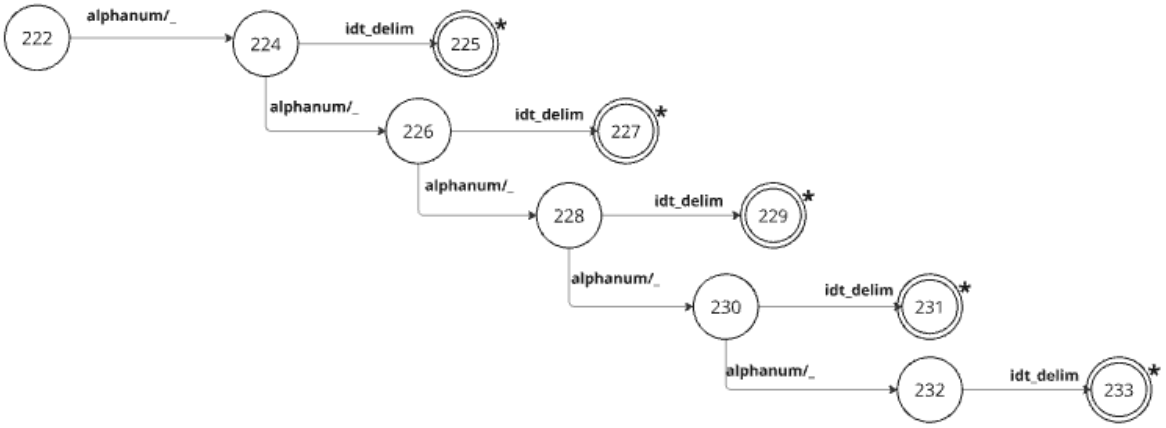


Identifiers

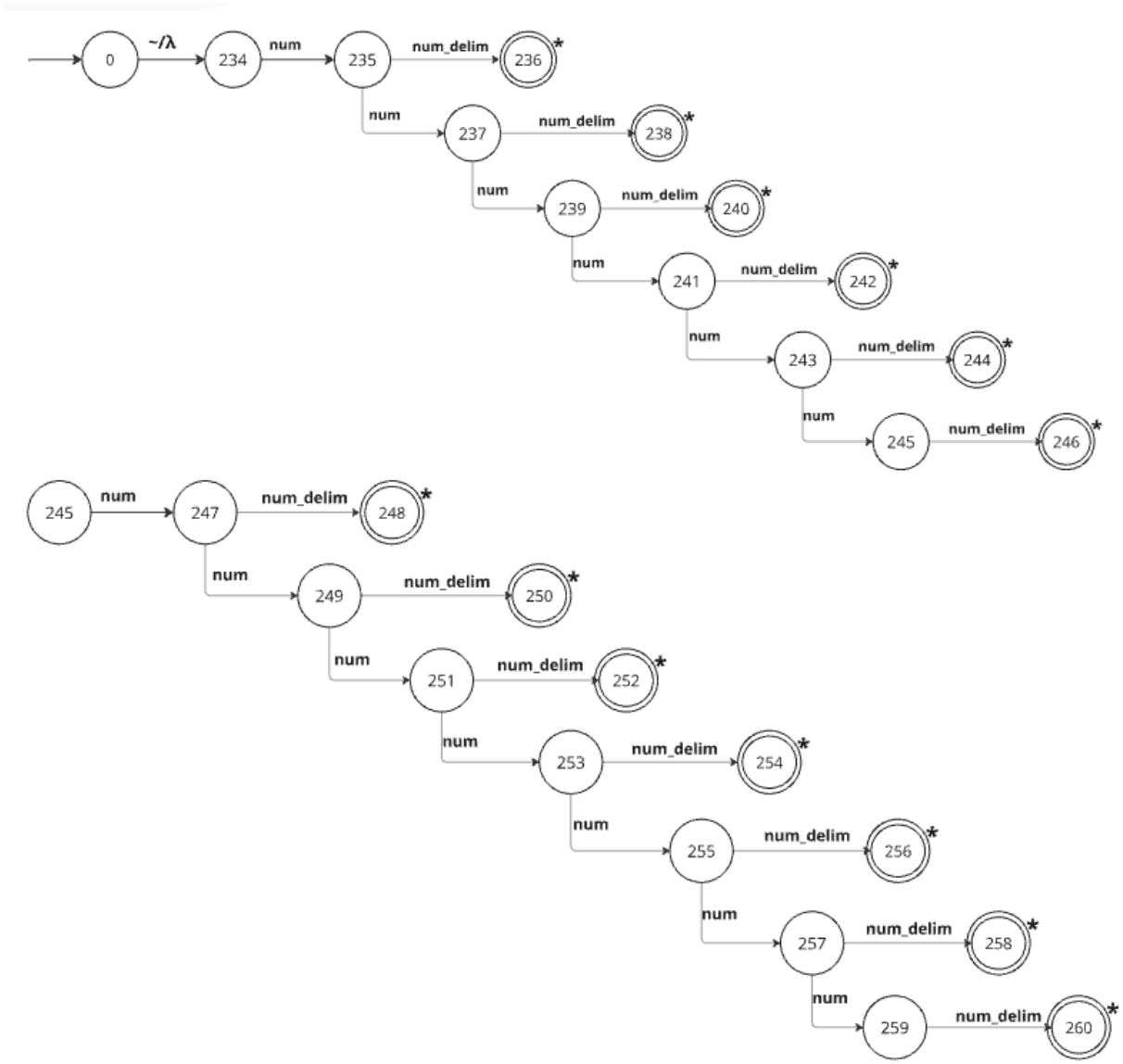

0
@
193
lowercase
194
idt_delim
195
alphanum/_
196
idt_delim
197
alphanum/_
198
idt_delim
199
alphanum/_
200
idt_delim
201
alphanum/_
202
idt_delim
203


202
alphanum/_
204
idt_delim
205
alphanum/_
206
idt_delim
207
alphanum/_
208
idt_delim
209
alphanum/_
210
idt_delim
211
alphanum/_
212
idt_delim
213

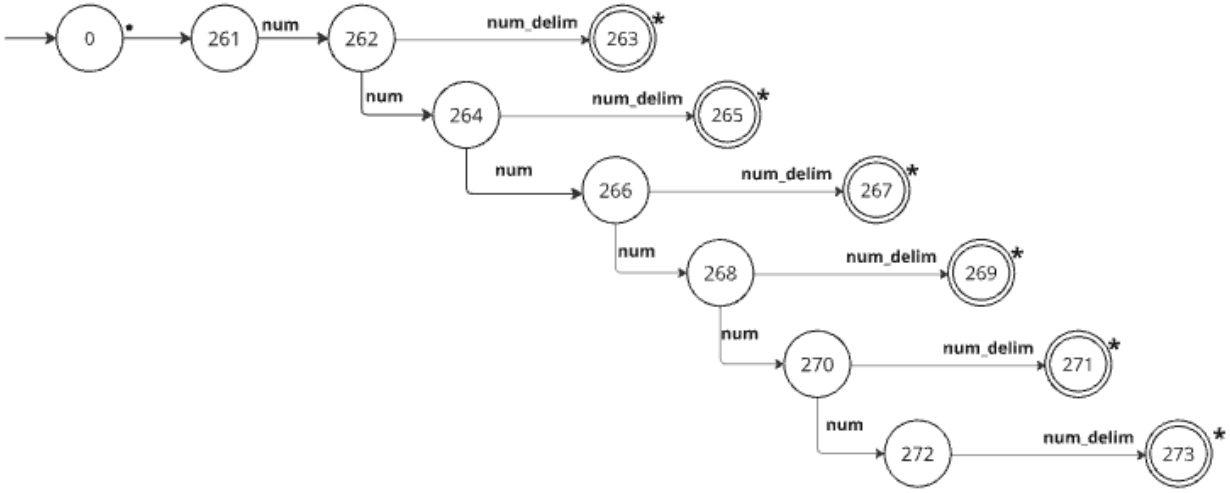

212
alphanum/_
214
idt_delim
215
alphanum/_
216
idt_delim
217
alphanum/_
218
idt_delim
219
alphanum/_
220
idt_delim
221
alphanum/_
222
idt_delim
223


222
alphanum/_
224
idt_delim
225
alphanum/_
226
idt_delim
227
alphanum/_
228
idt_delim
229
alphanum/_
230
idt_delim
231
alphanum/_
232
idt_delim
233

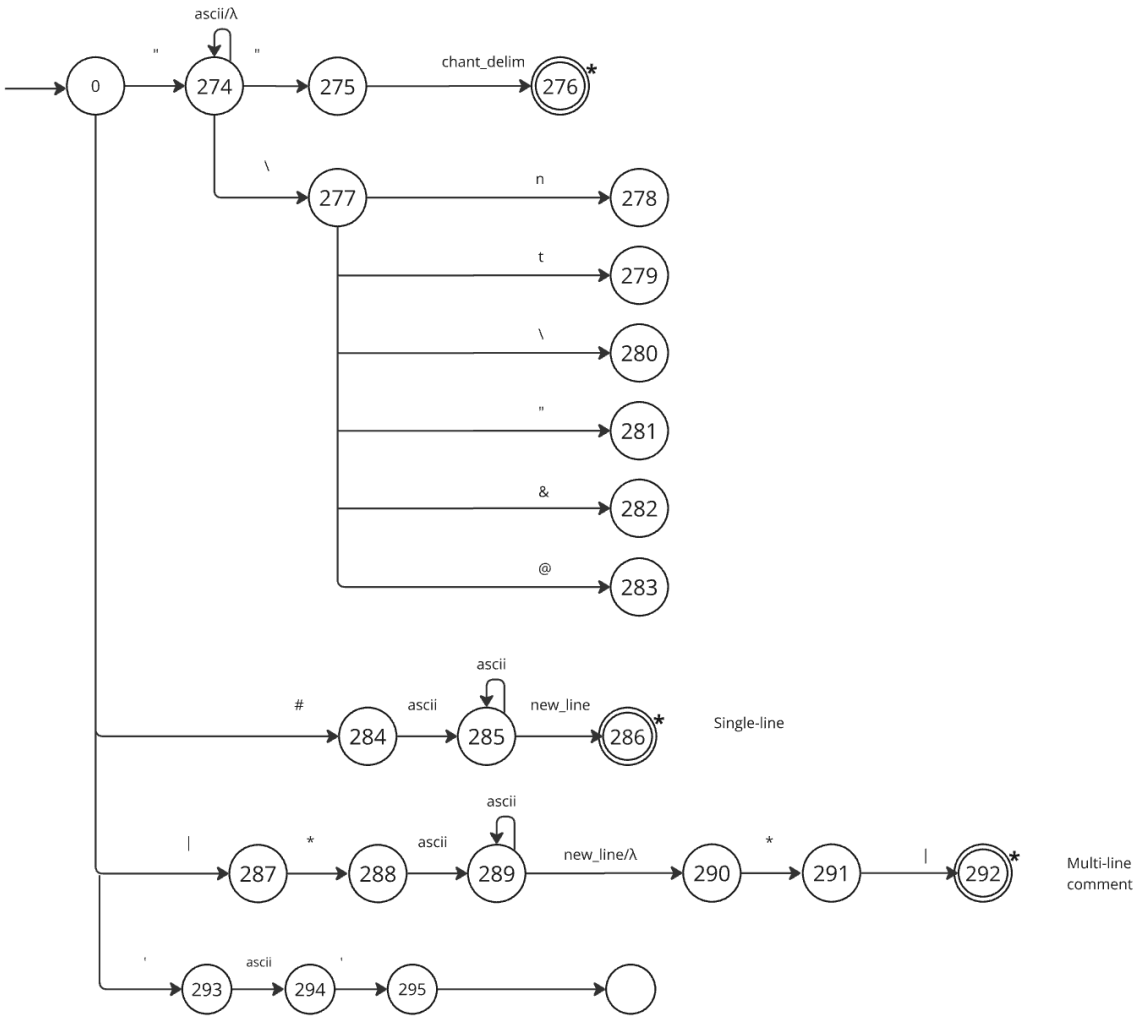
Numeric Literals (Positive/Negative Feint)



Floater Literals (Positive/Negative)



Comments and Chant



CONTEXT FREE GRAMMAR

1	<program>	→	<global_decl> <function_dec> <functions> Team() { <local_decl> <team_statements> }
2	<global_decl>	→	<global_declared> <global_decl>
3	<global_decl>	→	λ
4	<global_declared>	→	<variable_dec>
5	<global_declared>	→	<array_dec>
6	<global_declared>	→	<constant_dec>
7	<function_dec>	→	<function_decls> <function_dec>
8	<function_dec>	→	λ
9	<functions>	→	<function_definition> <functions>
10	<functions>	→	λ
11	<variable_dec>	→	<variable_decls>
12	<variable_decls>	→	<data_type> identifier <initialize_end> <decls_end>;
13	<decls_end>	→	, identifier <initialize_end> <decls_end>
14	<decls_end>	→	λ
15	<data_type>	→	Feint
16	<data_type>	→	Floater
17	<data_type>	→	Chant
18	<data_type>	→	Stat
19	<initialize_end>	→	= <global_value> <decls_end>
20	<initialize_end>	→	λ
21	<array_dec>	→	<data_type> <array_decls>
22	<array_decls>	→	identifier[<array_size>] <2d> <array_initialization>;

23	<2d>	→	λ
24	<2d>	→	[<array_size>]
25	<array_initialization>	→	= { <row> <rows> }
26	<array_initialization>	→	λ
27	<row>	→	{<array_elem> <elements> }
28	<rows>	→	, {<array_elem> <elements> }
29	<rows>	→	λ
30	<element_index>	→	Feint literal
31	<array_size>	→	Feint literal
32	<array_elem>	→	Feint literal
33	<array_elem>	→	Floater literal
34	<array_elem>	→	Chant literal
35	<elements>	→	, <array_elem><elements>
36	<elements>	→	λ
37	<constant_dec>	→	Const <data_type> identifier = <literal>;
38	<function_decls>	→	<data_type> identifier(<parameters>) <function_definition>;
39	<parameters>	→	<parameter><parameters_end>
40	<parameters>	→	λ
41	<parameter>	→	<data_type> identifier
42	<parameters_end>	→	, <parameters>
43	<parameters_end>	→	λ
44	<function_definition>	→	λ
	<function_definition>	→	{<team_statements> <rebound_statements>}
45	<function_definition>	→	Dunk identifier(<parameters>){<team_statements>}

46	<local_decl>	→	<local_declared> <local_decl>
47	<local_decl>	→	λ
48	<local_declared>	→	<variable_dec>
49	<local_declared>	→	<array_dec>
50	<rebound_statements>	→	Rebound <local_value>;
51	<rebound_statements>	→	λ
52	<team_statements>	→	<statement> <next_statements>
53	<team_statements>	→	<dribble_statements> <next_statements>
	<team_statements>	→	<array_dec><next_statements>
54	<next_statements>	→	<team_statements> <next_statements>
55	<next_statements>	→	λ
56	<statement>	→	<announce_statement>
57	<statement>	→	<inbound_statement>
58	<statement>	→	<forward_statement>
59	<statement>	→	<play_statement>
60	<statement>	→	<run_play_statement>
61	<statement>	→	<variable_dec>
62	<statement>	→	<array_dec>
63	<statement>	→	<assignment_statement>
64	<statement>	→	<switch_statement>
65	<statement>	→	<array_manipulation>
66	<array_manipulation>	→	identifier[<element_index>] <2d_manipulation> <array_man>;
67	<2d_manipulation>	→	[<element_index>]
68	<2d_manipulation>	→	λ

69	<array_man>	→	= <array_elem>
70	<array_man>	→	λ
71	<switch_statement>	→	Switch (<switch_exp>) {<case_list> <default_case>}
72	<switch_exp>	→	identifier
73	<switch_exp>	→	Feint literal
74	<switch_exp>	→	Chant literal
75	<case_list>	→	<case> <case_list>
76	<case>	→	Get <switch_value>:
77	<switch_value>	→	‘arith_op’
78	<switch_value>	→	‘alphabet’
79	<switch_value>	→	Feint literal
80	<switch_value>	→	Floater literal
81	<default_case>	→	Default: <Announce>
82	<assignment_statement>	→	<data_type> identifier <assign_op> <local_value>;
83	<inbound_statement>	→	Inbound(<input> <inputs>);
84	<input>	→	Chant literal
85	<inputs>	→	, identifier
86	<announce_statement>	→	Announce(<outputs>);
87	<outputs>	→	<output><output_end>
88	<output_end>	→	& <outputs>
89	<output_end>	→	λ
90	<output>	→	<global_value>
91	<output>	→	<local_value>
92	<output>	→	<chant_concat>
93	<global_value>	→	identifier

94	<global_value>	→	<literal>
95	<global_value>	→	<expression>
96	<local_value>	→	Identifier <arr_identifier>
97			
98			
99	<arr_identifier>	→	[<element_index>]
	<arr_identifier>		λ
100	<local_value>	→	<expression>
101	<local_value>	→	<func_call>
102	<local_value>	→	<type_casted>
103	<local_value>	→	<escape_sequence>
104	<escape_sequence>	→	\<esc_charac>
105	<esc_charac>	→	n
106	<esc_charac>	→	t
107	<esc_charac>	→	\
108	<esc_charac>	→	"
109	<esc_charac>	→	@
110	<esc_charac>	→	&
111	<literal>	→	Feint literal
112	<literal>	→	Floater literal
113	<literal>	→	Chant literal
114	<literal>	→	Stat literal
115	<type_casted>	→	<data_type> (<type_cast1>)
116	<type_cast1>	→	<literal>
117	<type_cast1>	→	identifier

118	<assign_op>	→	=
119	<assign_op>	→	+=
120	<assign_op>	→	-=
121	<assign_op>	→	*=
122	<assign_op>	→	%=
123	<assign_op>	→	/=
124	<arith_op>	→	+
125	<arith_op>	→	-
126	<arith_op>	→	*
127	<arith_op>	→	/
128	<arith_op>	→	%
129	<rel_op>	→	<
130	<rel_op>	→	>
131	<rel_op>	→	<=
132	<rel_op>	→	>=
133	<rel_op>	→	<relation_equality>
134	<relation_equality>	→	==
135	<relation_equality>	→	!=
136	<log_op>	→	AND
137	<log_op>	→	OR
138	<log_op>	→	NOT
139	<expression>	→	<arithmetic_expr>
140	<expression>	→	<relational_expr>
141	<expression>	→	<logical_expr>
142	<arithmetic_expr>	→	<term> <arithexpr_end>

143	<arithexpr_end>	→	<arith_op> <term> <arithexpr_end>
144	<arithexpr_end>	→	λ
145	<term>	→	identifier
146	<term>	→	<num_literal>
147	<term>	→	<num_type_casted>
148	<term>	→	(<arithmetic_expr>)
149	<num_literal>	→	Feint literal
150	<num_literal>	→	Floater literal
151	<relational_expr>	→	<rel_operand> <rel_op> <rel_operand> <rel_end>
	<rel_end>	→	<rel_op> <rel_operand> <rel_end>
152	<rel_end>	→	λ
153	<rel_operand>	→	<arithmetic_expr>
154	<rel_operand>	→	identifier
155	<rel_operand>	→	<type_casted>
156	<rel_operand>	→	<num_literal>
157	<rel_operand>	→	(<rel_expr>)
158	<rel_expr>	→	<rel_operand_eq> <relation_equality> <rel_operand_eq> <rel_end_eq>
159	<rel_end_eq>	→	<relation_equality> <rel_operand_eq> <rel_end_eq>
160	<rel_end_eq>	→	λ
161	<rel_operand_eq>	→	Stat literal
162	<rel_operand_eq>	→	Chant literal
163	<rel_operand_eq>	→	identifier
164	<rel_operand_eq>	→	(<relational_expr>)

165	<logical_expr>	→	<log_operand><log_op><log_operand> <log_end>
166	<log_end>	→	<log_op><log_operand><log_end>
167	<log_end>	→	λ
168	<log_operand>	→	<logical_expr>
169	<log_operand>	→	<relational_expr>
170	<log_operand>	→	identifier
171	<log_operand>	→	Stat literal
172	<log_operand>	→	(<logical_expr>)
173	<chant_concat>	→	<chant_operand> <chant_concat_end>
174	<chant_concat>	→	<concat_op> <concat_end>
175	<chant_operand>	→	Chant literal
176	<chant_operand>	→	identifier
177	<chant_operand>	→	<chant_type_casted>
178	<chant_type_casted>	→	Chant (<chant_typcst>)
179	<chant_typcst>	→	<literal>
180	<chant_typcst>	→	identifier
181	<chant_concat_end>	→	& <concat_op>
182	<chant_concat_end>	→	λ
183	<concat_end>	→	& <chant_operand>
184	<concat_end>	→	λ
185	<concat_op>	→	<chant_operand>
186	<concat_op>	→	<num_literal>
187	<concat_op>	→	identifier
188	<concat_op>	→	<num_type_casted>
189	<num_type_casted>	→	<num_data_type>(num_literal)

190	<num_data_type>	→	Feint
191	<num_data_type>	→	Floater
192	<dribble_statements>	→	Dribble (<condition>) { <team_statements> } <pass_statements> <shoot_scp>
193	<pass_statements>	→	<pass_scp> <pass_statements>
194	<pass_statements>	→	λ
195	<pass_scp>	→	Pass (<condition>) { <team_statements> }
196	<shoot_scp>	→	Shoot { <team_statements> }
197	<shoot_scp>	→	λ
198	<condition>	→	<relational_expr>
199	<condition>	→	<logical_expr>
200	<condition>	→	Stat literal
201	<forward_statement>	→	Forward (<forward_initialization>; <forward_condition>; <forward_incdec>) {<loop_stmts> <next_loopstmts>}
202	<forward_initialization>	→	<numdata_type> identifier = <initial_operand>
203	<numdata_type>	→	Feint
204	<initial_operand>	→	<num_literal>
205	<initial_operand>	→	<arithmetic_expr>
206	<forward_condition>	→	<forward_operand><forward_op><forward_operand>
207	<forward_operand>	→	identifier
208	<forward_operand>	→	<arithmetic_expr>
209	<forward_operand>	→	<num_literal>
210	<forward_op>	→	<rel_op>
211	<forward_incdec>	→	Identifier <forward_incdec_operand> <num_literal>
212	<forward_incdec_operand>	→	-=
213	<forward_incdec_operand>	→	+=

214	<loop_stmts>	→	<statement>
215	<loop_stmts>	→	<loop_dribble>
216	<loop_dribble>	→	Dribble (<condition>) { <loop_stmts> <next_loopstmts> } <loop_pass> <loop_shoot>
217	<loop_pass>	→	<next_loop_pass> <loop_pass>
218	<loop_pass>	→	λ
219	<next_loop_pass>	→	Pass (<condition>) { <loop_stmts> <next_loopstmts> }
220	<loop_shoot>	→	Shoot { <loop_stmts> <next_loopstmts> }
221	<loop_shoot>	→	λ
222	<next_loopstmts>	→	<loop_stmts> <next_loopstmts>
223	<next_loopstmts>	→	λ
224	<next_loopstmts>	→	Timeout;
225	<next_loopstmts>	→	Resume;
226	<play_statement>	→	Play (<condition>) { <loop_stmts> <next_loopstmts><forward_incdec>}
227	<run_play_statement>	→	Run { <loop_stmts> <next_loopstmts><forward_incdec> } Play (<condition>);
228	<func_call>	→	identifier(<arguments>);
229	<arguments>	→	<argument> <arguments_end>
230	<arguments_end>	→	,<arguments>
233	<argument>	→	<literal>
234	<argument>		Identifier <array_id>
235	<array_id>		λ
236	<array_id>		[<element_index>]
237	<argument>	→	<type_casted>

FIRST SET

First Set			
#	Production		Production Set
1	<program>	→	{ Feint, Floater, Chant, Stat, λ}
2	<global_decl>	→	{ Feint, Floater, Chant, Stat, Const, λ }
3	<global_declared>	→	{ Feint, Floater, Chant, Stat, Const }
4	<function_dec>	→	{ Feint, Floater, Chant, Stat, λ }
5	<functions>	→	{ Feint, Floater, Chant, Stat, Dunk, λ }
6	<variable_dec>	→	{ Feint, Floater, Chant, Stat}
7	<data_type>	→	{ Feint, Floater, Chant, Stat,}
8	<initialize_end>	→	{=, λ}
9	<array_dec>	→	{ Feint, Floater, Chant, identifier}
10	<2d>	→	{λ, []}
11	<array_initialization>	→	{=, λ}
12	<array_manipulation>	→	{identifier}
13	<2d_manipulation>	→	{λ, []}
14	<array_man>	→	{=, λ}
15	<element_index>	→	{Feint literal}
16	<array_size>	→	{Feint literal}
17	<array_elem>	→	{Feint literal, Floater literal, Chant literal}
18	<elements>	→	{, ,λ}
19	<constant_dec>	→	{ Const }
20	<function_decls>	→	{ Feint, Floater, Chant, Stat }
21	<parameters>	→	{ Feint, Floater, Chant, Stat, λ }
22	<parameters_end>	→	{, λ}
23	<function_definition>	→	{ λ, {, Dunk }
24	<local_decl>	→	{ Feint, Floater, Chant, Stat, λ }

25	<rebound_statements>	→	{ Rebound, λ }
26	<team_statements>	→	{ Announce, Inbound, Dribble, Play, Forward, Run, Switch, Feint, Floater, Chant, Stat, identifier }
27	<next_statements>	→	{ Announce, Inbound, Dribble, Play, Forward, Run, switch, Feint, Floater, Chant, Stat, identifier, λ }
28	<statement>	→	{ Announce, Inbound, identifier, Feint, Floater, Chant, Play, Run, Switch }
29	<switch_statement>	→	{ Switch }
30	<case_list>	→	{ Get }
31	<case>	→	{ Get }
32	<switch_value>	→	{ Feint literal, Floater literal, ‘}
33	<default_case>	→	{ Default }
34	<assignment_statement>	→	{ Feint, Floater, Chant, Stat}
35	<inbound_statement>	→	{ Inbound }
36	<input>	→	{ Chant literal}
37	<inputs>	→	{ , }
38	<announce_statement>	→	{ Announce }
39	<outputs>	→	{identifier, Feint literal, Floater literal, Chant literal, Stat literal, “\n, \t, \\, \”, \@, \&”, λ, }
40	<output_end>	→	{ &, λ }
41	<global_value>	→	{identifier, Feint literal, Floater literal, (, +, -, *, /, % , < , >, <=, >=, ==, !=, \n, \t, \\, \”, \@, \&, Chant literal, Stat literal}
42	<local_value>	→	{identifier, Feint literal, Floater literal, λ, (, +, -, *, /, %, Chant literal, Stat literal, \n, \t, \\, \”, \@, \&}
43	<escape_sequence>	→	{ \n, \t, \\, \”, \@, \& }
44	<literal>	→	{ Feint literal, Floater literal, Chant literal Stat literal }
45	<type_casted>	→	{ Feint, Floater, Chant, Stat}
46	<type_cast1>	→	{ identifier, Feint literal, Floater literal, Chant literal Stat literal}
47	<assign_op>	→	{=, +=, -=, *=, %=, /=, }

48	<arith_op>	→	{+, *, %, /, - }
49	<rel_op>	→	{<, >, <=, >=}
50	<relation_equality>	→	{==, !=}
51	<log_op>	→	{AND, OR, NOT}
52	<expression>	→	{identifier, Feint literal, Floater literal, Stat Literal, Feint, Floater,Chant, Stat (}
53	<num_literal>	→	{Feint literal, Floater literal}
54	<relational_expr>	→	{identifier, Feint, Floater, Chant, Stat, Feint literal, Floater literal, (, λ}
55	<rel_operand>	→	{identifier, Chant, Stat, Feint literal, Floater literal, Feint, Floater, (}
56	<rel_expr>	→	{identifier, Stat literal, Chant Literal, (}
57	<rel_operand_eq>	→	{Stat literal, Chant Literal, identifier, (}
58	<logical_expr>	→	{identifier, Stat literal, (, Chant, Feint literal, Floater literal, λ}
59	<log_operand>	→	{identifier, Stat literal, (, Chant, Feint literal, Floater literal, λ, <, >, <=, >=,}
60	<chant_concat>	→	{Chant literal, identifier, Chant, Feint literal, Floater literal, Stat Literal, Feint, Floater, &, λ, }
61	<dribble_statements>	→	{ Dribble }
62	<pass_statements>	→	{λ, Pass}
63	<shoot_scp>	→	{λ, Shoot}
64	<condition>	→	{identifier, Chant, Stat, Feint literal, Floater literal, (, λ, Stat literal}
65	<forward_statement>	→	{Forward}
66	<forward_initialization>	→	{Feint}
67	<initial_operand>	→	{Feint literal, Floater literal, identifier, Chant, Stat, (, }
68	<forward_condition>	→	{+=, -=, Feint literal, Floater literal, identifier, Chant, Stat, (,<, >, <=, >= }
69	<forward_operand>	→	{Feint literal, Floater literal, identifier, Chant, Stat,}
70	<forward_op>	→	{<, >, <=, >=}

71	<forward_incdec>	→	Identifier
72	<forward_incdec_operand>	→	{+=, -=}
73	<loop_stmts>	→	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, λ, }
74	<loop_dribble>	→	{Dribble}
75	<loop_pass>	→	{λ, Pass}
76	<loop_shoot>	→	{λ, Shoot}
77	<next_loopstmts>	→	{ λ,Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, Timeout; , Resume; }
78	<play_statement>	→	{Play}
79	<run_play_statement>	→	{Run}
80	<func_call>	→	{ identifier }
81	<arguments>	→	{Feint, Floater, Chant, Stat, Feint literal Floater literal, Chant literal, Stat literal, identifier}
82	<arguments_end>	→	{ “,”}
83	<argument>	→	{Feint, Floater, Chant, Stat, Feint literal Floater literal, Chant literal, Stat literal, identifier}
84	<array_decls>	→	{identifier}
85	<decls_end>	→	{, , λ}
86	<local_declared>	→	{ Feint, Floater, Chant, Stat}
87	<output>	→	{identifier, Feint literal, Floater literal, λ, (, +, -, *, /, %, Chant, Stat literal, \n, \t, \\, \”, \@, \&, Chant literal, &}
88	<term>	→	{identifier, Feint literal, Floater literal, Feint, Floater, (}
89	<log_end>	→	{AND, OR, NOT, λ}
90	<rel_end_eq>	→	{==, !=, λ}
91	<arithexpr_end>	→	{+, -, *, /, &,λ}
92	<concat_op>	→	{Chant literal, identifier, Chant, Feint literal, Floater literal, identifier, Feint, Floater}
93	<chant_operand>	→	{Chant literal, identifier, Chant}

94	<chant_type_casted>	→	{Chant}
95	<num_type_casted>	→	{Feint, Floater}
96	<concat_end>	→	{&, λ}
97	<chant_concat_end>	→	{&, λ}
98	<variable_decls>	→	{ Feint, Floater, Chant, Stat,}
99	<parameter>	→	{Feint, Floater, Chant, Stat,}
100	<arithmetic_expr>	→	{identifier, Feint literal, Floater literal, Feint, Floater, (}
101	<num_data_type>	→	{Feint, Floater}
102	<rel_end>	→	{λ}
103	<pass_scp>	→	{Pass}
104	<next_loop_pass>		{Pass}
105	<rows>		{, , λ}
106	<esc_charac>		{n, t, \, ", @, &}

FOLLOW SET

Follow Set			
#	Production		
1	<program>	→	
2	<global_decl>	→	{ Feint, Floater, Chant, Stat, Dunk }
3	<global_declared>	→	{ Feint, Floater, Chant, Stat, Const }
4	<function_dec>	→	{ Feint, Floater, Chant, Stat, Dunk }
5	<functions>	→	{Team(), Feint, Floater, Chant, Stat, Dunk }
6	<variable_dec>	→	{ Feint, Floater, Chant, Stat, Const }
7	<data_type>	→	{ identifier, (}
8	<initialize_end>	→	{, , ; }
9	<array_dec>	→	{ Feint, Floater, Chant, Stat, Const }
10	<2d>	→	{ =, ;}
11	<array_initialization>	→	{;}
12	<array_manipulation>	→	{ Inbound, Announce, Dribble, Play, Forward, identifier }
13	<2d_manipulation>	→	{=, ;, Feint literal, Floater literal, Chant literal, Inbound, Announce, Dribble, Play, Forward, identifier }
14	<array_man>	→	{;}
15	<element_index>	→	{] }
16	<array_size>	→	{] }
17	<array_elem>	→	{, , } }
18	<elements>	→	{ } }
19	<constant_dec>	→	{ Feint, Floater, Chant, Stat, Const }
20	<function_decls>	→	{ Feint, Floater, Chant, Stat }
21	<parameters>	→	{) }
22	<parameters_end>	→	{) }

23	<function_definition>	→	{ Feint, Floater, Chant, Stat, Dunk, ; }
24	<local_decl>	→	{ }, Inbound, Announce, Dribble, Play, Forward, identifier}
25	<rebound_statements>	→	{ } }
26	<team_statements>	→	{ }, Inbound, Announce, Dribble, Play, Forward, identifier }
27	<next_statements>	→	{ }, Inbound, Announce, Dribble, Play, Forward, identifier, Switch, Feint, Floater, Run }
28	<statement>	→	{ Announce, Inbound, Dribble, Play, Forward, Run, switch, Feint, Floater, Chant, Stat, identifier, } }
29	<switch_statement>	→	{ Inbound, Announce, Dribble, Play, Forward, identifier }
30	<case_list>	→	{ Get }
31	<case>	→	{ Get }
32	<switch_value>	→	{ : }
33	<default_case>	→	{ } }
34	<assignment_statement>	→	{ Announce, Inbound, Dribble, Play, Forward, Run, switch, Feint, Floater, Chant, Stat, identifier, } }
35	<inbound_statement>	→	Announce, Inbound, Dribble, Play, Forward, Run, switch, Feint, Floater, Chant, Stat, identifier, } }
36	<input>	→	{ , }
37	<inputs>	→	{) }
38	<announce_statement>	→	{ Announce, Inbound, Dribble, Play, Forward, Run, switch, Feint, Floater, Chant, Stat, identifier, } }
39	<outputs>	→	{&,)}
40	<output>	→	{&,)}
41	<output_end>	→	{&,)}
42	<global_value>	→	{&,)}
43	<local_value>	→	{&,)}
44	<escape_sequence>	→	{&,)}
45	<literal>	→	{; , &,)}

46	<type_casted>	→	{&,), Feint, Floater, Chant, Stat, Feint literal Floater literal, Chant literal, Stat literal, identifier, “,”}
47	<type_cast1>	→	{) }
48	<assign_op>	→	{identifier, Feint literal, Floater literal, Chant literal, ;, (, +, -, *, /, %, Chant, Stat literal, \n, \t, \\, \”, \@, \& }
49	<arith_op>	→	{ identifier, Feint literal, Floater literal, Feint, Floater, ‘}
50	<rel_op>	→	{ Feint literal, Floater literal, identifier, Chant, Stat, (}
51	<relation_equality>	→	{ Feint literal, Floater literal, identifier, Chant literal, Stat literal, (}
52	<log_op>	→	{ identifier, Stat literal, (, Chant, Feint literal, Floater literal, <, >, <=, >=, AND, OR, NOT }
53	<expression>	→	{ &,) }
54	<num_literal>	→	{=, +=, -=, *=, %=, /=, +, =, *, /, %, identifier, Feint literal, Floater literal, (,λ,), } , Chant literal, identifier, Chant, Stat, identifier, Feint, Floater <, >, <=, >=, ; ==, !=, }
55	<relational_expr>	→	{&,), ==, != }
56	<rel_operand>	→	{<, >, <=, >=, identifier, Chant, Stat, Feint literal, Floater literal, (, λ, <, >, <=, >=, }
57	<rel_expr>	→	{identifier, Chant, Stat, Feint literal, Floater literal, (, }
58	<rel_operand_eq>	→	{==, != }
59	<logical_expr>	→	{AND, OR, NOT, identifier, Chant, Stat, Feint literal, Floater literal, (, λ, <, >, <=, >=, Stat literal, &,)}
60	<log_operand>	→	{AND, OR, NOT}
61	<chant_concat>	→	{&,)}
62	<dribble_statements>	→	{ Inbound, Announce, Dribble, Play, Forward, identifier, }, }
63	<pass_statements>	→	{ Inbound, Announce, Dribble, Play, Forward, identifier, } , Shoot}
64	<shoot_scp>	→	{Inbound, Announce, Dribble, Play, Forward, identifier, }, }
65	<condition>	→	{) }
66	<forward_statement>	→	{ Inbound, Announce, Dribble, Play, Forward, identifier }

67	<forward_initialization>	→	{;}
68	<initial_operand>	→	{;}
69	<forward_condition>	→	{;}
70	<forward_operand>	→	{<, >, <=, >=, ; }
71	<forward_op>	→	{Feint literal, Floater literal, identifier, Chant, Stat, }
72	<forward_incdec>	→	{), } }
73	<forward_incdec_operand>	→	{Feint literal, Floater literal }
74	<loop_stmts>	→	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, λ, Timeout; , Resume;, }, }
75	<loop_dribble>	→	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, , Timeout; , Resume; }
76	<loop_pass>	→	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, λ, Timeout; , Resume; Shoot}
77	<loop_shoot>	→	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, , Timeout; , Resume;}
78	<next_loopstmts>	→	{}, Identifier, }
79	<play_statement>	→	{ Inbound, Announce, identifier, Feint, Floater, Chant, Play, Run, Switch }
80	<run_play_statement>	→	{ Inbound, Announce, identifier, Feint, Floater, Chant, Play, Run, Switch }
81	<func_call>	→	{identifier, Feint literal, Floater literal, λ, (, +, -, *, /, %, Chant, Stat literal, “\n, \t, \\, \”, \@, \&”}
82	<arguments>	→	{), } #AddFollowSetOfArguments_end
83	<arguments_end>	→	{}
84	<term>	→	{=, +=, -=, *=, %=, /=, +, =, *, /, %, identifier, Feint literal, Floater literal, (, &, <, >, <=, >=, Chant, Stat }
85	<arithmetic_expr>	→	{ &,), =, +=, -=, *=, %=, /=, +, =, *, /, %, identifier, Feint literal, Floater literal, (, <, >, <=, >=, Chant, Stat, }
86	<concat_op>	→	{&,)}

87	<array_decls>	→	{ Feint, Floater, Chant, Stat, Const }
88	<decls_end>	→	{, , ;}
89	<local_declared>	→	{ Feint, Floater, Chant, Stat, identifier}
90	<log_end>	→	{&,)}
91	<rel_end_eq>	→	{identifier, Chant, Stat, Feint literal, Floater literal, (, }
92	<argument>	→	{Feint, Floater, Chant, Stat, Feint literal Floater literal, Chant literal, Stat literal, identifier, “,”}
93	<num_type_casted>	→	{=, +=, -=, *=, %=, /=, +, =, *, /, %, identifier, Feint literal, Floater literal, (, &, <, >, <=, >=, Chant, Stat }
94	<chant_operand>	→	{&,)}
95	<chant_type_casted>	→	{&,)}
96	<concat_end>	→	{&,)}
97	<chant_concat_end>	→	{&,)}
98	<variable_decls>	→	{ Feint, Floater, Chant, Stat, Const }
99	<parameter>	→	{, λ}
100	<rel_end>	→	{&,), ==, != , }
101	<num_data_type>	→	{ (}
102	<pass_scp>	→	(Pass, Shoot}
103	<next_loop_pass>	→	{Pass}
104	<arithexpr_end>	→	{ &,), =, +=, -=, *=, %=, /=, +, =, *, /, %, identifier, Feint literal, Floater literal, (, <, >, <=, >=, Chant, Stat}
105	<rows>		{ } }
106	<esc_charac>		{&,)}

PREDICT SET

1	First<global_decl> <function_dec> <functions> Team() { <local_decl> <team_statements> }	First(<global_decl>)	{ Feint, Floater, Chant, Stat, Const, λ }
2	First<global_declared> <global_decl>	First(<global_declared>)	{ Feint, Floater, Chant, Stat, Const }
3	First (λ) ∪ Follow <global_decl>	Follow(<global_decl>)	{ Feint, Floater, Chant, Stat, Dunk }
4	First<variable_dec>	First(<variable_dec>)	{ Feint, Floater, Chant, Stat}
5	First<array_dec>	First(<array_dec>)	{ Feint, Floater, Chant, identifier}
6	First<constant_dec>	First(<constant_dec>)	{ Const }
7	First<function_decls><function_dec>	First(<function_decls>)	{ Feint, Floater, Chant, Stat }
8	First (λ) ∪ Follow <function_dec>	Follow(<function_dec>)	{ Feint, Floater, Chant, Stat, λ }
9	First <function_definition> <functions>	First(<function_definition>)	{ Feint, Floater, Chant, Stat, Dunk }
10	First (λ) ∪ Follow <functions>	Follow(<functions>)	{Team(), Feint, Floater, Chant, Stat, Dunk }
11	First <variable_decls>	First(<variable_decls>)	
12	First <data_type> identifier <initialize_end> <decls_end>;	First(<data_type>)	{ Feint, Floater, Chant, Stat,}
13	First , identifier <initialize_end> <decls_end>	First(,)	{ , }
14	First (λ) ∪ Follow <decls_end>	Follow(<decls_end>)	{, , ;}
15	First Feint	First(Feint)	[Feint }
16	First Floater	First(Floater)	{ Floater }
17	First Chant	First(Chant)	{ Chant }
18	First Stat	First(Stat)	{ Stat }
19	First = <global_value> <decls_end>	First(=)	{ = }

20	First (λ) ∪ Follow <initialize_end>	Follow(<initialize_end>)	{, , ;, }
21	First <data_type> <array_decls>	First(<data_type>)	{ Feint, Floater, Chant, Stat,}
22	First (identifier[<array_size>] <2d> <array_initialization>;)	First(identifier)	{ Identifier }
23	First(λ) ∪ Follow<2d>	Follow(<2d>)	{=, Feint literal, Floater literal, Chant literal}
24	First([<array_size>])	First([<array_size>])	{Feint literal}
25	First = { <row> <rows> }	First(=)	{ = }
26	First (λ) ∪ Follow <array_initialization>	Follow(<array_initialization>)	{;}
27	First {<array_elem> <elements> }	First(<array_elem>)	{Feint literal, Floater literal, Chant literal}
28	First , {<array_elem> <elements> }	First(,)	{ , }
29	First (λ) ∪ Follow <rows>	Follow(<rows>)	{ rows }
30	First Feint literal	First(Feint literal)	{ Feint literal }
31	First Feint literal	First(Feint literal)	{ Feint literal }
32	First Feint literal	First(Feint literal)	{ Feint literal }
33	First Floater literal	First(Floater literal)	{ Floater literal }
34	First Chant literal	First(Chant literal)	{ Chant literal }
35	First , <array_elem><elements>	First(,)	{ , }
36	First (λ) ∪ Follow <elements>	Follow(<elements>)	{ , }
37	First Const <data_type> identifier = <literal>;	First(Const)	{ Const }
38	First <data_type> identifier(<parameters>);	First(<data_type>)	{ Feint, Floater, Chant, Stat,}
39	First <parameter><parameters_end>	First(<parameter>)	{Feint, Floater, Chant, Stat,}
40	First (λ) ∪ Follow <parameters>	Follow(<parameters>)	{) }
41	First <data_type> identifier	First(<data_type>)	{ Feint, Floater, Chant, Stat,}
42	First , <parameters>	First(,)	{ , }

43	First (λ) ∪ Follow<parameters_end>	Follow(<parameters_end>)	{) }
44	First <data_type> identifier(<parameters>) {<team_statements> <rebound_statements>}	First(<data_type>)	{ Feint, Floater, Chant, Stat,}
45	First Dunk identifier(<parameters>){<team_statements>}	First(Dunk)	{ Dunk }
46	First <local_declared> <local_decl>	First(<local_declared>)	{ Feint, Floater, Chant, Stat, identifier}
47	First (λ) ∪ Follow <local_decl>	Follow <local_decl>	{ Feint, Floater, Chant, Stat, identifier}
48	First <variable_dec>	First(<variable_dec>)	{ Feint, Floater, Chant, Stat}
49	First <array_dec>	First(<array_dec>)	{ Feint, Floater, Chant, Stat, Const }
50	First Rebound <local_value>;	First(Rebound)	{ Rebound }
51	First (λ) ∪ Follow<rebound_statements>	Follow(<rebound_statements>)	{ } }
52	First <statement> <next_statements>	First(<statement>)	{Announce, Inbound, identifier, Feint, Floater, Chant, Play, Run, Switch }
53	First <dribble_statements> <next_statements>	First(<dribble_statements>)	{ Inbound, Announce, Dribble, Play, Forward, identifier, }, }
54	First <team_statements> <next_statements>	First(<team_statements>)	{Announce, Inbound, Dribble, Play, Forward, Run, switch, Feint, Floater, Chant, Stat, identifier }
55	First (λ) ∪ Follow <next_statements>	Follow(<next_statements>)	{ Inbound, Announce, Dribble, Play, Forward, identifier }
56	First <announce_statement>	First(<announce_statement>)	{Announce}
57	First <inbound_statement>	First(<inbound_statement>)	{Inbound}
58	First <forward_statement>	First(<forward_statement>)	{ Inbound, Announce, Dribble, Play, Forward, identifier }
59	First <play_statement>	First(<play_statement>)	{Play}

60	First <run_play_statement>	First(<run_play_statement>)	{Run}
61	First <variable_dec>	First(<variable_dec>)	{ Feint, Floater, Chant, Stat}
62	First <array_dec>	First(<array_dec>)	{ Feint, Floater, Chant, Stat, Const }
63	First <assignment_statement>	First(<assignment_statement>)	{ Feint, Floater, Chant, Stat,}
64	First <switch_statement>	First(<switch_statement>)	{Switch}
65	First <array_manipulation>	First(<array_manipulation>)	{identifier}
66	First identifier[<element_index>] <2d_manipulation> <array_man>;	First(identifier)	{identifier}
67	First [<element_index>]	First(<element_index>)	{Feint literal}
68	First (λ) ∪ Follow <2d_manipulation>	Follow(<2d_manipulation>)	{=, Feint literal, Floater literal, Chant literal, Inbound, Announce, Dribble, Play, Forward, identifier }
69	First = <array_elem>	First(=)	{ = }
70	First (λ) ∪ Follow <array_man>	Follow(<array_man>)8	{;}
71	First Switch (<switch_exp>) {<case_list> <default_case>}	First<Switch>	{ Switch }
72	First identifier	First identifier	{ Identifier }
73	First Feint literal	First Feint literal	{ Feint literal }
74	First Chant literal	First Chant literal	{ Chant literal }
75	First <case> <case_list>	First <case>	{Get}
76	First Get <switch_value>:	First Get	{Get}
77	First ‘arith_op’	First<arith_op>	{=, +=, -=, *=, %=, /=, +, =, *, /, %, }
78	First ‘alphabet’	First<alphabet>	{ alphabet }
79	First Feint literal	First(Feint literal)	{ Feint literal }
80	First Floater literal	First(Floater literal)	{ Floater literal }
81	First Default: <Announce>	First(Default)	{ Default }

82	First <data_type> identifier <assign_op> <local_value>;	First(data_type>)	{ Feint, Floater, Chant, Stat,}
83	First Inbound(<input> <inputs>);	First(Inbound>)	{ Inbound }
84	First Chant literal	First(First Chant literal)	{ Chant literal}
85	First , identifier	First(,)	{ , }
86	First Announce(<outputs>);	First(Announce)	{ Announce}
87	First <output><output_end>	First(output)	{ Output}
88	First & <outputs>	First(&)	{ & }
89	First (λ) ∪ Follow <output_end>	Follow(output_end)	{&,)}
90	First <global_value>	First(global_value)	{identifier, Feint literal, Floater literal, λ, (, +, -, *, /, % , < , >, <=, >=, ==, !=, \n, \t, \\, \", \@, \&, Chant, Stat literal}
91	First <local_value>	First(local_value)	{identifier, Feint literal, Floater literal, λ, (, +, -, *, /, %, Chant, Stat literal, \n, \t, \\, \", \@, \&}
92	First <chant_concat>	First(chant_concat)	{Chant literal, identifier, Chant, Feint literal, Floater literal, &, λ, }
93	First identifier	First(identifier)	{ identifier }
94	First <literal>	First(<literal>)	{ Literal }
95	First <expression>	First(<expression>)	{identifier, Feint literal, Floater literal, (}
96	First loc_identifier	First(loc_identifier)	{ loc_identifier }
97	First arr_identifier	First(arr_identifier)	{ arr_identifier }
98	First identifier	First(identifier)	{ identifier }
99	First identifier [<element_index>]	First(identifier)	{ identifier }
100	First <expression>	First(<expression>)	{identifier, Feint literal, Floater literal, (}
101	First <func_call>	First(<func_call>)	{ identifier }
102	First <type_casted>	First(<type_casted>)	{Feint, Floater, Chant, Stat}

103	First <escape_sequence>	First (<escape_sequence>)	{\n, \t, \\, \", \@, \&}
104	First "\<esc_charac>"	First("\<esc_charac>")	{ \esc_charac }
105	First n	First(n)	{ n }
106	First t	First(t)	{ t }
107	First \	First(\)	{ \ }
108	First "	First(")	{ " }
109	First @	First(@)	{ @ }
110	First &	First(&)	{ & }
111	First Feint literal	First(Feint literal)	{ Feint literal }
112	First Floater literal	First(Floater literal)	{ Floater literal }
113	First Chant literal	First(Chant literal)	{ Chant literal }
114	First Stat literal	First(Stat literal)	{ Stat literal }
115	First <data_type> (<type_cast1>)	First(<data_type>)	{ Feint, Floater, Chant, Stat,}
116	First <literal>	First(<literal>)	{ literal }
117	First identifier	First(identifier)	{ Identifier }
118	First =	First(=)	{ = }
119	First +=	First(+=)	{ += }
120	First -=	First(-=)	{ -= }
121	First *=	First(*=)	{ *= }
122	First %=	First(%=)	{ %= }
123	First /=	First(/=)	{ /= }
124	First +	First(+)	{ + }
125	First -	First(-)	{ - }
126	First *	First(*)	{ * }
127	First /	First(/)	{ / }
128	First %	First(%)	{ % }
129	First <	First(<)	{ < }

130	First >	First(>)	{ > }
131	First <=	First(<=)	{ <= }
132	First >=	First(>=)	{ >= }
133	First <relation_equality>	First(<relation_equality>)	{==, !=}
134	First ==	First(==)	{ == }
135	First !=	First(!=)	{ != }
136	First AND	First(AND)	{ AND }
137	First OR	First(OR)	{ OR }
138	First NOT	First(NOT)	{ NOT }
139	First <arithmetic_expr>	First(<arithmetic_expr>)	{ Feint, Floater }
140	First <relational_expr>	First(<relational_expr>)	{identifier, Chant, Stat, Feint literal, Floater literal, (, λ, <, >, <=, >=, }
141	First <logical_expr>	First(<logical_expr>)	{identifier, Stat literal, (, Chant, Feint literal, Floater literal, λ, <, >, <=, >=,}
142	First <term> <arithexpr_end>	First(<term>)	{identifier, Feint literal, Floater literal, Feint, Floater, (}
143	First <arith_op> <term> <arithexpr_end>	First(<arith_op>)	{=, +=, -=, *=, %=, /=, +, =, *, /, %, }
144	First (λ) ∪ Follow <arithexpr_end>	Follow(<arithexpr_end>)	{=, +=, -=, *=, %=, /=, +, =, *, /, %, identifier, Feint literal, Floater literal, (}
145	First identifier	First(identifier)	{ Identifier }
146	First <num_literal>	First(<num_literal>)	{Feint literal, Floater literal}
147	First <num_type_casted>	First(<num_type_casted>)	{Feint, Floater}
148	First <arithmetic_expr>	First(<arithmetic_expr>)	{ Feint, Floater }
149	First Feint literal	First(Feint literal)	{Feint literal}
150	First Floater literal	First(Floater literal)	{Float literal}
151	First <rel_operand> <rel_op> <rel_operand> <rel_end>	First(<rel_operand)	{identifier, Chant, Stat, Feint literal, Floater literal,

			(, }
152	First (λ) ∪ Follow<rel_end>	Follow(<rel_end>)	{ rel_end }
153	First <arithmetic_expr>	First(<arithmetic_expr>)	{identifier, Feint literal, Floater literal, Feint, Floater, (}
154	First identifier	First(identifier)	{ Identifier }
155	First <type_casted>	First(<type_casted>)	{Feint, Floater, Chant, Stat, λ}
156	First <num_literal>	First(<num_literal>)	{Feint literal, Floater literal}
157	First (<rel_expr>)	First(<rel_expr>)	{identifier, Stat literal, Chant, ==, !=, (, λ }
158	First (<rel_operand_eq> <relation_equality> <rel_operand_eq> <rel_end_eq>)	First(<rel_operand_eq>)	{Stat literal, Chant, identifier, (}
159	First (<relation_equality> <rel_operand_eq> <rel_end_eq>)	First (<relation_equality>)	{==, !=}
160	First (λ) ∪ Follow <rel_end_eq>	Follow(<rel_end_eq>)	{identifier, Chant, Stat, Feint literal, Floater literal, (, }
161	First Stat literal	First(Stat literal)	{ Stat }
162	First Chant literal	First(Chant literal)	{ Chant }
163	First identifier	First(identifier)	{ identifier }
164	First <relational_expr>	First(<relational_expr>)	{identifier, Chant, Stat, Feint literal, Floater literal, (, λ, <, >, <=, >=, }
165	First <log_operand><log_op><log_operand> <log_end>	First(<log_operand>)	{identifier, Stat literal, (, Chant, Feint literal, Floater literal, λ, <, >, <=, >=,}
166	First <log_op><log_operand><log_end>	First(<log_op>)	{AND, OR, NOT}
167	First (λ) ∪ Follow <log_end>	Follow(<log_end>)	{AND, OR, NOT, λ}
168	First <logical_expr>	First(<logical_expr>)	{identifier, Stat literal, (, Chant, Feint literal, Floater literal, λ, <, >, <=, >=,}
169	First <relational_expr>	First(<relational_expr>)	{identifier, Chant, Stat, Feint literal, Floater literal,

			(, λ, <, >, <=, >=, }
170	First identifier	First(identifier)	{ identifier }
171	First Stat literal	First(Stat literal)	{ Stat literal }
172	First (<logical_expr>)	First(<logical_expr>)	{identifier, Stat literal, (, Chant, Feint literal, Floater literal, λ, <, >, <=, >=,}
173	First (<chant_operand> <chant_concat_end>)	First(<chant_operand>)	{Chant literal, identifier, Chant}
174	First <concat_op> <concat_end>	First(<concat_op>)	{Chant literal, identifier, Chant, Feint literal, Floater literal, identifier, Feint, Floater}
175	First Chant literal	First(Chant literal)	{ Chant literal }
176	First identifier	First(identifier)	{ Identifier }
177	First <chant_type_casted>	First(<chant_type_casted>)	{ Chant }
178	First Chant (<chant_typcst>)	First(Chant)	{ Chant }
179	First <literal>	First(<literal>)	{ literal }
180	First identifier	First(identifier)	{ identifier }
181	First & <concat_op>	First(&)	{&}
182	First (λ) ∪ Follow <chant_concat_end>	Follow(<chant_concat_end>)	{&,)}
183	First & <chant_operand>	First(&)	{&}
184	First (λ) ∪ Follow <concat_end>	Follow(<concat_end>)	{&,)}
185	First <chant_operand>	First(<chant_operand>)	{Chant literal, identifier, Chant}
186	First <num_literal>	First(<num_literal>)	{Feint literal, Floater literal}
187	First identifier	First(identifier)	{ Identifier }
188	First <num_type_casted>	First(<num_type_casted>)	{Feint, Floater}
189	First <num_data_type>(num_literal)	First(<num_data_type>)	{Feint, Floater}
190	First Feint	First(Feint)	{ Feint }
191	First Floater	First(Floater)	{ Floater }

192	First Dribble (<condition>) { <team_statements> } <pass_statements> <shoot_scp>	First(Dribble)	{ Dribble }
193	First <pass_scp> <pass_statements>	First(<pass_scp>)	{Pass}
194	First (λ) ∪ Follow <pass_statements>	Follow(<pass_statements>)	{ Inbound, Announce, Dribble, Play, Forward, identifier, } , Shoot}
195	First Pass (<condition>) { <team_statements> }	First(Pass)	{λ, Pass}
196	First Shoot { <team_statements> }	First(Shoot)	{ Shoot }
197	First (λ) ∪ Follow <shoot_scp>	Follow(<shoot_scp>)	{Inbound, Announce, Dribble, Play, Forward, identifier, }, }
198	First <relational_expr>	First(<relational_expr>)	{identifier, Chant, Stat, Feint literal, Floater literal, (, λ, <, >, <=, >=, }
199	First <logical_expr>	First (<logical_expr>)	{identifier, Stat literal, (, Chant, Feint literal, Floater literal, λ, <, >, <=, >=,}
200	First Stat literal	First(Stat literal)	{ Stat literal }
201	First (Forward <forward_initialization>; <forward_condition>; <forward_incdec>) {<loop_stmts> <next_loopstmts>}	First(Forward)	{ Forward }
202	First <num_data_type> identifier = <initial_operand>	First(<num_data_type>)	{Feint, Floater}
203	First Feint	First(Feint)	{ Feint literal }
204	First <num_literal>	First(<num_literal>)	{Feint literal, Floater literal}
205	First <arithmetic_expr>	First(<arithmetic_expr>)	{identifier, Feint literal, Floater literal, Feint, Floater, (}
206	First <forward_operand><forward_op><forward_operand>	First(<forward_operand>)	Feint literal, Floater literal, identifier, Chant, Stat,}
207	First identifier	First(identifier)	{ Identifier }
208	First <arithmetic_expr>	First(<arithmetic_expr>)	{identifier, Feint literal, Floater literal, Feint, Floater, (}

209	First <num_literal>	First(<num_literal>)	{Feint literal, Floater literal}
210	First <rel_op>	First(<rel_op>)	{<, >, <=, >=}
211	First Identifier <forward_incdec_operand> <num_literal>	First(Identifier)	{ Identifier }
212	First +=	First(+=)	{ += }
213	First -=	First(-=)	{ -= }
214	First <statement>	First(<statement>)	{Announce, Inbound, identifier, Feint, Floater, Chant, Play, Run, Switch }
215	First <loop_dribble>	First(<loop_dribble>)	{Dribble}
216	First Dribble (<condition>) { <loop_stmts> <next_loopstmts> } <loop_pass> <loop_shoot>	First(Dribble)	{ Dribble }
217	First <next_loop_pass> <loop_pass>	First(<next_loop_pass>)	{ Pass }
218	First (λ) ∪ Follow <loop_pass>	Follow(<loop_pass>)	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, λ, Timeout; , Resume; Shoot}
219	First Pass (<condition>) { <loop_stmts> <next_loopstmts> }	First(Pass)	{ Pass }
220	First Shoot { <loop_stmts> <next_loopstmts> }	First(Shoot)	{Shoot}
221	First (λ) ∪ Follow <loop_shoot>	Follow(<loop_shoot>)	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, , Timeout; , Resume;}
222	First <loop_stmts> <next_loopstmts>	First(<loop_stmts>)	{Inbound, Announce, identifier, Feint, Floater, Chant, Forward, Play, Run, Switch, Dribble, λ, }
223	First (λ) ∪ Follow <next_loopstmts>	Follow(<next_loopstmts>)	{ Identifier }
224	First Timeout;	First(Timeout;)	{Timeout}
225	First Resume;	First(Resume;)	{Resume}
226	First Play (<condition>) { <loop_stmts>	First(Play)	{Play}

	<next_loopstmts><forward_incdec>}		
227	First Run { <loop_stmts> <next_loopstmts><forward_incdec> } Play (<condition>);	First(Run)	{Run}
228	First identifier(<arguments>);	First(identifier)	{Identifier}
229	First <argument> <arguments_end>	First(<argument>)	{Feint, Floater, Chant, Stat, Feint literal Floater literal, Chant literal, Stat literal, identifier}
230	First ,<arguments>	First(<”,”>)	{ , }
231	First arg_identifier	First(arg_identifier)	{arg_identifer}
232	First array_identifier	First(array_identifier)	{array_identifer}
233	First <literal>	First(<literal>)	{Feint literal, Floater literal, Chant literal Stat literal}
234	First <type_casted>	First(<type_casted>)	{Feint, Floater, Chant, Stat, λ}
235	First identifier	First(identifier)	{Identifier}
236	First identifier [<element_index>]	First(identifier)	{Identifier}